knowledge_type: book
title: Mars Product Reference Handbook
title_zh: Mars 产品参考手册
client: mars
edition: "2026.1"
author: 数砚公司
classification: 数砚机密
handling: 仅限数砚公司内部授权人员使用。未经数砚公司书面许可，禁止以任何形式对外复制、传播、披露或提供。非授权人员如误获本文件，应立即停止使用，并永久删除或销毁其持有、存储或控制的全部副本（含打印件、缓存及衍生副本）。违反上述规定的，数砚公司将依据适用法律、合同约定及公司制度追究相关人员责任。
status: published
last_updated: 2026-10-27
generated_by: tools/ps/build_reference_handbook_editions.py
source_manifest: editorial/manuscript-manifest.json
source_sha256:
"appendix/a-product-objects-and-identifiers.md":
"105dc343a84f1fb6f24c86bfecb004fbe71d7633650e948e2a00da3800a8f339"
"appendix/b-systems-apis-and-data-chains.md":
"dd4dd64fb53a558521de5154717eee10aa49b8fdfc80153daf4001c80b10f803"
"appendix/c-product-project-workbench.md":
"5b0a6643d2e5c1299383fa7194137413ee6cfda7340e3f0ec980ae9ec9d601bd"
"appendix/d-product-open-questions-and-maintenance.md":
"fff4603c537b64f827ed519e93882eb18ff7086bbbbee0c4f1c34f9bde8e99db"
"book/01-why-a-product-has-many-names.md":
"b0fde82e368459c02c6188a72e84732d84591c800162fa08938d0432e22c3ce7"
"book/02-product-identity-chain.md":
"b62c02cf8542374b84ebd597fb5fa3092588b4e61effe30c35038509afc6eef1"
"book/03-product-hierarchy-and-classification.md":
"8d25270e624941bb3b32066dbe86c6b6178228e08444c9c174ae2a8a43a05ef8"
"book/04-product-time-and-lifecycle.md":
"68989a0a69d00e4b46408c7725c80c914ac1dde6d6933fd63836ed04e0ec4fda"
"book/05-bundles-package-units-and-conversion.md":
"eb8e9cb6d6c8216dacf8713043bf5de2343d08993eab98aff0c238843b86e367"
"book/06-product-write-integration-and-query.md":
"a93717cf4eee8178f8dcb97b0ac24633f587b75cff859b3d24c69730ccf25fef"
"book/07-pricing-chain.md":
"696c6d85bc2d81863ceccc6ea60ae5f0ec44bcb148509ec31a4bec7384c6f541"
"book/08-product-in-store-distribution-and-perfect-store.md":
"8d9daf16da6859456724d9d16c392e55f9b4a56b6a528fc72ad17276e8835422"
"book/09-product-in-dtr-order-inventory-and-reports.md":
"d277b4e04f4a616d58c15a3335ab942c9c5626e80e271f543b2f8af84a5766a8"
"book/10-data-consistency-same-name-multiple-code.md":
"a2975efd865ec061bd4ca5c06cbdd851cd55c94d3b9bfee76ad711bf04a00d64"
"book/11-product-change-impact-and-regression.md":
"dd4ea1562fc303888a5ca48d150f9898bc2ec3993037cd0ec42e088122eb2cc4"

"book/12-product-projects-operations-and-evolution.md":
"1ce27a10dc87686168de666ff6b4b3f03bd62649ea7027f40ba679bd6c1e383c"

Mars 产品参考手册

作者： 数砚公司

密级： 数砚机密

使用限制： 仅限数砚公司内部授权人员使用。未经数砚公司书面许可，禁止以任何形式对外复制、传播、披露或提供。非授权人员如误获本文件，应立即停止使用，并永久删除或销毁其持有、存储或控制的全部副本（含打印件、缓存及衍生副本）。违反上述规定的，数砚公司将依据适用法律、合同约定及公司制度追究相关人员责任。

版本 `2026.1` **状态** 已发布（附已披露缺口）

产品身份、单位、时间、价格与跨系统消费

阅读契约

先按第 1–12 章建立完整主线，再用 A–D 案头卷查对象、系统、项目和缺口。正文中的 staging 事实均受观察时间约束。

本装配稿由章节和案头卷确定性生成。单项事实仍以各章 source_docs、claim ledger、文档、代码、staging 部署与 staging database 为准。

目录

- 第 1 章 一个商品为什么有多个名字
- 第 2 章 产品身份链：material、zrep、nickname 与 barcode
- 第 3 章 产品层次：业务板块、品类、品牌、子品类与属性
- 第 4 章 产品时间：创建、上市、调整、退市与历史版本
- 第 5 章 组合产品、包装单位与换算
- 第 6 章 产品主数据的写入、集成与查询
- 第 7 章 价格链：从价格文件到可用价格
- 第 8 章 产品怎样进入门店分销与完美门店
- 第 9 章 产品怎样进入经销商、订单、库存与报表
- 第 10 章 产品一致性：同名、多码、单位、时点与投影
- 第 11 章 产品变更影响分析与回归
- 第 12 章 产品项目、运维与演进
- 第 A 卷 产品对象与标识
- 第 B 卷 产品系统与数据链
- 第 C 卷 产品项目工作台
- 第 D 卷 产品事实缺口、术语与维护

概念与别名速查

本表由 `editorial/concept-registry.json` 生成，用于检索规范名、英文名和别名；正文仍须首次使用时解释概念。

规范名	英文名与别名	不等于	最短定义
产品	Product / 商品	品牌、品类、昵称、条码、S2 组合	在 Mars 业务系统中被产品主数据引用和消费的商品对象。
物料代码	material_code / materialCode / material	zrep_code、nickname_code、barcode	来自 ERP 的物料代码，用于识别物料。
物料代表码	zrep_code / zrepCode / zrep	不带单位的 material、产品昵称	带物料单位语境的代表码。
产品昵称	Product Nickname / nickname / 产品别名	自然名称主键、物料身份、条码	业务用于引用产品的可读名称及其稳定代码。
产品条码	Barcode / TDU bar code / MCU bar code / RSU bar code	产品主键、包装单位	某个包装或销售载体上的可扫描标识，必须结合层级解释。
产品单位	Unit of Measure / UOM / TDU / MCU / RSU / CS / SB / PCE	订单交易单位、报表聚合单位	表达配送、销售与零售单元及其换算的业务单位。
S2 组合产品	S2 Bundle / 组合产品 / Bundle	单个 material、订单组合	由 Header、Line、Unit 三层表达的 Mars 组合商品。
成品物料	Finished Good / WS finished good / 成品	产品全集、条码投影	来自 ERP WS 成品清单、承载包装与条码的物料记录。
产品数据集成投影	Product DI Projection / product DI / di_otr_* / DI 投影	主数据事实源、完整源表副本	面向下游集成消费而生成的产品投影。
产品事实源边界	Product Source-of-Truth Boundary / 产品 SoT / 双产品体系	任一查询表、DI 投影	某类产品事实拥有权威写入和当前状态的体系边界，当前双体系仍未最终统一。
价格批次	Price Batch / pricing batch / batch_id	单行价格结果	价格文件从上传到结果写入的全链路追踪单元。
价格版本历史	Price Version History / pricing_final_data_history / append-only history	当前结果行	每次写入价格结果时追加保留的历史版本。
经销商产品	Distributor Product / dtr_sku / DTR product	Mars product、经销商可售状态	DTR/经销商系统中的产品记录，需要映射回 Mars 产品。
当前有效产品记录	Current Valid Product Record / current product / 当前产品	任意非删除行、生命周期 Active	按目标时点、有效期、删除和对象状态共同判定的产品记录。
产品投影	Product Projection / 产品查询面 / product query projection	产品事实源	面向查询、集成或报表生成的产品派生表面。

第 1 章 一个商品为什么有多个名字

一张门店报表把某个商品称为“巧克力”，一张分销规则使用产品昵称，一条订单记录使用物料编码，一箱货的外包装又有一个条码。四个系统都在描述“同一个产品”，但没有任何一个字段天然承载全部含义。

这不是命名不统一的小问题。项目如果按名称匹配，会把同名商品合并；按物料匹配，会忽略同一物料的不同包装单位；按条码匹配，会漏掉没有条码或条码尚未维护的商品；按昵称匹配，又会把一段业务描述误当成稳定对象身份。

本章的中心问题因此不是“产品叫什么”，而是：**面对一条商品事实时，应该先确认哪一层身份?**

为了说明冲突，先使用一个匿名复合案例：门店要求补一个巧克力 SKU，运营人员按昵称找到“看起来正确”的产品，下单人员却按另一个物料代码录入，报表随后在同一家店出现两行相近但不相同的 SKU。这个案例不对应任何单一真实项目。它提醒读者，名称相似只是问题出现的入口，真正需要检查的是商品语义、物料、规格单位、昵称和扫描载体。

1. 产品不是一个名称字段

业务人员常用品牌名、商品名或包装描述沟通，因为这些词最接近货架和消费者。系统则需要可以稳定关联的关系键。两者服务于不同任务。

“德芙”“士力架”“M&M's”是品牌，“巧克力”“冰淇淋”是品类语境，一箱 24 盒或一盒 12 支则是包装和单位。一个物料在 ERP 中有物料编码；产品中心还可能给业务分析人员一个昵称。外包装和销售单位也可能各自拥有条码。

这些词可以同时正确，却不能互相替代。一个品牌下有很多产品，一个产品可能有多个包装单位，一个单位可能有多个条码来源，一个昵称又可能被多个带单位的物料记录共享。

因此，产品身份不是“最像商品名的那个字段”，而是一组有明确作用域的关系。

2. materialCode、zrepCode、nicknameCode 和 barcode 各回答什么

产品中心文档把三个核心标识分开解释：

- `materialCode` 标识 ERP 语境中的物料；
- `zrepCode` 表示带物料单位的物料代表码；
- `nicknameCode` 是昵称的稳定标识。

条码则回答“这个包装或销售单位能否被扫描识别”。条码与昵称、成品和单位投影存在关联，但条码不是产品对象本身。

可以用四个问题区分：

问题	首选标识	不能直接替代
这是哪一种物料	`materialCode`	商品名、昵称、条码
这是该物料的哪个单位规格	`zrepCode` + `zrepUom`	单独的物料代码
业务分析和规则怎样引用可读产品	`nicknameCode` / nickname	物料主键、条码
哪个销售包装可被扫描	barcode	产品对象、物料主键

这张表不是要求所有接口都接收四类键。它要求每一次查询先说明自己要解决哪一层问题，再看当前接口是否提供了足够的作用域。

3. 代码里确实同时保存这些层

当前本地代码仓 `mw-bmp/product-center/re-product-nickname` 的分支 `feature/pricing`、HEAD `f00bd1e` 中，`BzProduct` 实体对应 `bz_product` 表，并声明了 `zrep_code`、`zrep_uom`、`nickname`、`nickname_code`、`material_code`、品类、品牌和子品类等字段。

这证明产品记录的同一条表行并非只保存一个名称。它同时承载业务昵称、物料标识、单位代表码和分类引用。职责相近并不意味着它们可以互换：它们只是被放在同一读模型中，供不同问题使用。

昵称写入也体现了稳定标识与名称的分离。当前 `ProductNikeNameServiceImpl` 在昵称基表不存在目标昵称时，会通过 ID 生成器创建新的昵称代码并保存昵称值；更新产品昵称时，代码会关闭旧记录的有效期并插入一条新记录。

这说明“改昵称”不是简单覆盖一处文本。它可能产生新的历史版本，并且需要回答历史报表当时看到的是哪个名称。代码证明了这一实现路径，但具体生产使用范围仍需结合部署和消费方证据。

4. staging 数据证明这些层不是一一对应

2026-10-26 22:48:27 CST，staging `product_nickname.bz_product` 的精确行数为 3,115，`bz_product_nickname` 为 1,858，`t1_product_individual_current` 为 4,605，S2 Header 为 24，代码映射为 6,645。

这些数字不应用来做业务增长或在售判断，因为 staging 不是生产，表也不是同一语义层。它们只证明同一系统中的产品、昵称、单品、组合和编码映射分别有独立载体。

同一时点的最小业务键样本还显示：

```
material_code 60035935 → zrep_code 618562 → zrep_uom 1*1*1 → nickname_code 11404
material_code 80017791 → zrep_code 610410 → zrep_uom 1*1*20 → nickname_code 空
```

第二条记录尤其重要。它没有 `nickname_code`，但这不能直接推出“这个产品不存在”或“它在业务上不可用”。它只说明在当前 staging 的当前样本里，物料和单位记录可以先存在，而昵称字段不保证同时有值。

如果项目按昵称做内连接，这类记录可能消失；如果按产品存在判断可售，又会把另一个问题混进来。下游必须分别声明是否要求昵称、是否要求条码、是否要求可售和是否要求有效规则。

5. 名称相同不保证对象相同，名称缺失也不保证对象不存在

产品名称首先服务阅读和搜索。它可能来自 ERP 物料描述、业务昵称、品牌简称或报表展示名。名称可以重复，也可能为空或发生变化。

因此，名称至少承担三种角色：

1. 人类识别；
2. 模糊搜索；
3. 某些业务流程中的业务可读标签。

它不天然承担稳定主键。项目若用名称关联订单与规则，必须说明名称相同是否经过规范化、是否包含单位、是否可能在历史中变化，以及是否存在同名不同码。

反过来，昵称或条码缺失也不能直接消除物料身份。缺失只说明该层数据在当前载体和时间点不可用。系统可以存在一个还未完成业务维护或投影刷新的物料。

这正是产品排障和门店排障经常得出不同答案的原因：门店侧可能通过昵称或成品投影查找，产品侧则有更上层的物料和编码映射。先选错入口，就会在不同数据范围里寻找“同一个产品”。

6. 产品层次也不等于对象身份

产品文档使用业务板块、品类组、品牌和子品类描述业务组合。分类回答“这是什么类型的商品”“用于什么分析口径”，身份回答“这条记录究竟指向哪个产品和单位”。

下面三种归类都不能替代产品身份：

- 把品牌当产品，会让所有同品牌包装共享同一身份；
- 把品类当产品，会把不同品牌和物料混成一条；
- 把子品类当产品，则无法区分具体规格。

反过来，也不能认为分类只是展示标签。分类会影响品牌分析、分销规则、报表维度和合规边界。但分类关系是产品对象上的业务语义或引用，不应取代物料与单位键。

另一个必须保留的边界是：公司品牌目录不等于中国市场当前在售清单。品牌在册只能说明业务组合中存在该品牌，当前 SKU、包装、可售和必铺必须回到各自权威。

7. 为什么“当前”二字必须有时间

产品记录不是只读常量。昵称会更新，单位会调整，组合关系会变化，产品状态会演进，投影也会按其刷新节奏更新。

当前代码中的昵称更新会关闭旧有效行、插入新行。这个行为意味着“产品叫什么”本身就是时间相关事实。查询如果不带有效期或观察时间，就把多个版本压成了一个当前答案。

项目至少要区分：

- 事实在该时刻是什么；
- 当前表此刻是什么；
- 某个投影最近何时刷新；
- 报表统计时实际读取了哪个版本；
- 代码支持哪些历史查询方式。

本章的 staging 观察全部带 `2026-10-26` 时间点。它们不是今天的永久事实，也不能直接表示生产当前值。

8. 一个可执行的判断顺序

遇到“产品对不上”时，可以按以下顺序判断：

1. 先确认讨论的是品牌、品类、物料、单位规格、昵称还是条码；

2. 如果涉及交易或库存，找到 material 或 zrep，并确认单位；
3. 如果涉及扫描，找到条码及其所属单位，不把条码当主键；
4. 如果涉及规则或报表，确认其产品范围使用 nickname 还是其它键；
5. 如果涉及历史，带上有效期、期次或观察时间；
6. 如果各入口结论不同，先检查查询层和投影时点，再判断数据错误；
7. 无法确认时把问题保留为 unknown，不用名称相似来闭合。

这条顺序能减少一类常见错误：拿到一个名称后立即搜索所有表，找到第一条“看起来正确”的记录就开始做影响分析。搜索结果只能作为线索，稳定关系仍要通过业务键、时间和适用范围确认。

9. 本章留下的边界

本章只建立多标识分层，不回答以下问题：

- `bz_product` 与 `t1/t2/t3_*` 两套数据体系的最终 SoT；
- UOM 的权威归属和所有换算规则；
- 全球品牌目录与中国市场在售清单的完整映射；
- 价格链、可售范围、经销商映射和订单拆零的完整链路；
- staging 镜像与本地代码 HEAD 的精确提交对应关系。

这些问题会进入后续章节和案头 D。保留它们不是写作未完成，而是证据尚未允许给出确定结论。

案头入口与下一章

对象定义、字段卡和当前标识基数可在第 A 卷查阅，未决边界见第 D 卷。下一章继续拆解产品身份链，说明 material、zrep、nickname 与 barcode 为什么必须按不同作用域共同解释，而不能争夺一个万能标识。

第 2 章 产品身份链：material、zrep、nickname 与 barcode

上一章说明一个商品会以多个名字出现。本章继续往下追：当系统必须把订单、库存、规则和报表合成一条事实时，哪一个键足够？

一个直觉答案是“找到唯一产品主键”。但产品中心当前同时保存 material、zrep、昵称和条码，它们回答的问题不同。更困难的是，staging 当前数据并不满足任意两个标识的简单一一对应。项目如果先假定某个键全局唯一，后续查询、对账和变更分析就会从一个错误公理开始。

本章的中心问题是：**material、zrep、nickname 和 barcode 怎样组成一条可解释的产品身份链，而不是争夺一个万能主键？**

匿名复合案例：订单人员按 material 查到一个产品，库存人员用 zrep 查询却得到多条记录，门店报表又按 nickname 聚合。三个人都输入“正确编码”，结果数量却不同。案例不对应任何单一真实项目。实际原因可能是指标粒度、历史版本或数据异常；在没有完整业务键和时间窗口前，不能先说是“系统重复”。

1. 四类标识的职责

产品中心文档把标识分成不同职责：

标识	文档语义	适合回答	不能单独回答
material	ERP 物料代码	这是哪一种物料	哪个包装规格、哪个昵称、哪个条码
zrep	带物料单位的代表码	物料在什么包装单位语境下出现	业务昵称、订单组合、全部条码
nickname	业务可读产品/别名及其稳定代码	规则、报表或业务分析如何引用产品	实际物料、包装规格的完整事实
barcode	可扫描标识	这个包装/销售载体能否被识别	产品对象的唯一身份

职责不同意味着查询接口也必须声明作用域。一个接口接收 material，不等于返回一个单品；一个报表按 nickname 汇总，也不等于同一 nickname 只有一个 material。

2. 当前 staging 不是一对一

2026-10-26 22:57 CST，当前有效 `product_nickname.bz_product` 观察到：

- 3,115 行；
- 3,115 个 distinct material；
- 1,355 个 distinct zrep；
- 1,247 个 distinct nickname；
- 320 行 nickname_code 为空。

这不是四张表互相独立的说明，而是同一条 `bz_product` 记录同时保存这些字段后的实际基数。当前快照中，多个 material 可以共享同一个 zrep，也可以共享同一个 nickname；部分记录没有 nickname。

当前有效 `product_nickname.bz_ws_finished_good` 又观察到：

- 3,105 行；
- 3,105 个 distinct material；
- 1,350 个 distinct zrep；
- 208 行 TDU 条码为空；
- 320 行 RSU 条码为空。

它和 `bz_product` 的规模接近，但不能据此认为两张表一一对应。两者名称、字段、有效期和用途不同，差值还可能是历史、过滤或维护状态造成的。

3. 为什么 zrep 不能独立承担全链身份

文档把 zrep 解释为带物料单位的代表码，适合表达某种包装单位语境。当前 staging 中，一个 zrep 可以关联多个 material。抽样中：

zrep_code	distinct material_code
`376023`	32
`376030`	30
`376027`	26
`376020`	22

此外还有 209 个 material 的 zrep 为空。这些结果不能直接说明哪套规则错误，但足以排除“只要拿到 zrep 就能唯一确定 material”的无条件假设。

代码侧，`BzProduct` 和 `BzWsFinishedGood` 都声明 material 与 zrep 字段，`BzWsFinishedGood` 还声明 TDU、MCU、RSU 条码和每 TDU 的包装换算字段。表结构支持一端是多条包装或规格记录，因此查询必须说明自己要的是物料、包装规格、单位还是条码。

4. 条码是扫描载体，不是产品主键

条码投影 `product_sequoia.di_otr_nickname_barcodes` 在 2026-10-26 22:57 CST 有 1,646 行，对应 1,646 个 distinct nickname，但只有 1,583 个 distinct barcode 字符串。

因此，当前快照里条码字符串并不能独立唯一标识 nickname。一个条码字段也可能承载复合值或历史聚合；本次只做了字符串级 `COUNT(DISTINCT)`，没有证明条码的业务拆分规则。

同时，当前成品表有 208 行缺少 TDU 条码、320 行缺少 RSU 条码。没有条码不等于物料不存在，也不等于商品不可用；它只说明当前表没有提供这一扫描载体。

项目使用条码时必须继续问：

1. 条码属于哪个包装单位；
2. 条码是扫描原值还是多个条码的拼接投影；
3. 查询是否过滤删除和有效期；
4. 重复条码是历史、不同商品共用，还是数据异常；

5. 下游需要识别“商品”还是“这箱货”。

5. nickname 是业务语言，不是自然主键

nickname 让业务人员可以用较短名称引用产品，也方便规则和报表展示。但当前有 320 行没有 nickname_code，说明昵称不是每条产品记录的必填身份。

昵称代码由写入路径生成或引用；当前代码在昵称基表不存在时创建代码，更新昵称时关闭旧版本并插入新版本。这进一步说明昵称有维护生命周期。

使用 nickname 的安全方式是把它作为**已声明的业务范围键**：

- 如果只统计有昵称的产品，先说明过滤；
- 如果要求规则必须覆盖所有产品，先检查 nickname 缺失；
- 如果名称发生变化，带有效期或版本；
- 如果同一昵称对应多个 material，明确聚合目标和单位。

6. 不能用一个连接解决所有问题

常见错误是写一条大 SQL，把 material、zrep、nickname、barcode 四张表全部内连接，然后把结果当作产品全集。只要任一环节是多对一、空值或历史版本，内连接就会丢记录或放大行数。

更稳妥的顺序是：

```
确定业务问题
  → 选事实粒度
  → 选主业务键
  → 补必要维度
  → 声明时间窗口
  → 检查空值与一对多
  → 再连接辅助标识
```

不同问题首先需要选择不同的主粒度：

- 库存问题可能使用 material + zrep + 仓库；
- 规则问题可能使用门店 + nickname；
- 扫描问题使用图片或扫描事件 + barcode；
- 订单问题使用订单行 + 交易产品。

主粒度确定后，辅助标识只用于补充解释，不再反过来决定事实行数。

7. 一个可复算的产品身份卡

产品项目应至少记录：

项	示例问题
业务对象	物料、包装规格、昵称、条码还是订单组合
主键	当前查询用哪个字段定位事实
作用域	应用、渠道、客户、单位或业务域
时间	有效期、期次或观察时点
基数	候选是否预期一对一，哪些允许一对多
空值	哪一层允许缺失，缺失时业务如何处理
历史	关闭旧行、版本追加还是原位覆盖
对账键	两个下游怎样证明说的是同一事实

一张身份卡没有回答这八项时，所谓“统一产品编码”只是字段名一致，不是对象语义一致。

8. 本章能支持的设计判断

基于当前证据，可以支持以下判断：

1. 不能用商品名称作为稳定产品身份；
2. 不能假设 material、zrep、nickname、barcode 全部一一对应；
3. 条码必须结合包装单位和扫描原值解释；
4. nickname 缺失和条码缺失都是业务查询必须显式处理的状态；
5. 每项跨系统对账都要声明业务粒度、主键和时间窗口。

本章不能支持以下推断：

- 当前 zrep 多对一都是错误数据；
- 当前 1,247 个昵称就是中国市场完整在售产品数；
- 空条码都应由产品中心补录；
- 产品 SoT 已由两张产品表自动统一；
- staging 分布与生产相同。

这些结论只有在确认真实业务规则、数据结构来源和运行消费后才可以成立。

案头入口与下一章

需要查询字段含义、标识基数和未决边界时，进入第 A 卷和第 D 卷。下一章转入产品分类与层次，解释品牌、品类和子品类为何能组织分析语境，却不能取代对象身份。

第 3 章 产品层次：业务板块、品类、品牌、子品类与属性

产品分类常被当成一排下拉框：业务板块、品类、品牌、子品类。多个字段都在“描述产品”，项目于是容易顺手用其中一个字段代替产品身份，或者把字典中的全部品牌写成当前在售商品。

分类回答“这是什么、用于哪类分析”，身份回答“具体指的是哪条产品记录”。同一个产品可以属于多个分类维度，一个分类下也有很多产品。混淆两者，会让过滤、规则和报表在不知不觉中改变分母。

本章的中心问题是：**产品层次怎样支持业务理解，又为什么不能代替产品对象身份？**

1. 产品文档中的层次

产品实体文档给出两层业务视角：

- 业务板块描述公司层面的业务组合；
- 品类组及其品牌描述消费者可见的品牌与具体品类。

在产品中心的技术模型中，又可以看到品类、品牌、子品类和属性等分类字段。它们不是同一套组织方式，但共同为业务分析、规则筛选和报表展示提供语境。

分类树存在多个入口，不等于每个入口都是一级父子链。业务板块可能是分析口径，品类与子品类可能是产品分类，品牌又跨越品类。项目应先确认目标系统采用哪一套分类表，而不是从名称推出层级。

2. 分类字典不等于当前使用范围

2026-10-26 23:01 CST，staging 当前观察到：

- `bz_product_category` 有 8 条未删除品类字典；
- `bz_alias_brand` 有 31 条未删除品牌字典；
- 当前有效 `bz_product` 有 3,115 行；
- 这些产品实际使用 6 个品类名称、24 个品牌名称和 8 个子品类。

字典中有 8 个品类，不代表当前每条产品都填充品类；字典中有 31 个品牌，也不代表这 31 个品牌当前都有产品。

当前还有 320 行产品的 `product_category_name` 与 `alias_brand_name` 同时为空。它们仍存在于产品表，说明产品记录可以暂未完成分类语境。

因此，以下说法都不成立：

- 品类字典有 8 行，所以当前有 8 类在售产品；
- 品牌字典有 31 行，所以报表应出现 31 个品牌；
- 产品没有品类，所以产品对象不存在；
- 当前使用 24 个品牌，所以全球品牌组合只有 24 个。

准确表达必须带对象范围：**当前 staging 的有效产品行使用了多少分类值。**

3. 全球品牌组合不是中国当前 SKU 清单

业务实体文档明确说明，品牌列表覆盖公司层面的业务组合，不代表所有品牌都在中国市场上市或在售。实际在售 SKU、品类组和品牌组合回到中国上市清单或产品主数据。

这条边界对产品手册尤其重要。品牌目录适合理解品牌关系和常见别名，不适合：

- 生成当前建议铺货范围；
- 判断某 SKU 是否可售；
- 计算中国市场产品数量；
- 代替产品中心的有效产品记录。

分类可以解释“这个产品更可能属于哪里”，可售和铺货必须回到交易与规则事实。

4. 分类字段改变的是分析语境

当产品品类或品牌发生变化时，即使 material、zrep 和条码不变，报表中的品类汇总、品牌份额、分销规则产品范围也可能变化。

这说明分类是产品对象上的重要业务事实，却不是对象身份。一个人给产品改名、换品牌归类或补子品类，通常仍在描述同一个产品；分类变化需要版本和影响分析，不应新建产品身份。

反过来，两个 material 属于同一品牌，也不证明它们是同一个 SKU。品牌是共享分类节点，产品身份指向具体物料与单位。

5. 分类在规则与报表中的两种用法

5.1 作为筛选条件

规则或活动可能选择某品类、品牌、子品类下的产品范围。此时分类值帮助圈定候选集合，具体规则还要继续确定产品粒度、门店范围和生效期。

5.2 作为聚合维度

报表按品类或品牌分组，从而汇总销量、分销或执行结果。此时分类决定分组，不能反过来推断每一组只有一个产品。

同一个字段在筛选和聚合中的要求并不相同。筛选关心允许值是否完整，聚合关心映射是否稳定、是否允许空值、历史变更如何处理。

6. 为什么 320 条空分类需要进入设计

空分类不一定意味着数据错误。它可能表示：

- 产品尚未完成业务维护；
- 该产品不属于当前分类体系；
- 分类只在其它系统或其它层维护；
- 分类字段在当前表仅部分填充；
- 记录是特殊包装或非销售产品。

在没有业务规则和来源之前，不能选择其中一种解释。

项目能做的是显式处理空值：

1. 报表是否单列“未分类”；
2. 规则是否允许无品类产品命中；
3. 产品切换分类时是否同时关闭旧分类关系；
4. 总盘与分类明细能否对账；
5. 新增产品是否要求分类完成才能进入某些下游。

这些问题比把空值临时改填成“其它”更重要。一个假的分类值会永久污染后续分析，而空值至少保留“未知”。

7. 多套分类之间的映射

产品可能同时进入：

- 业务板块；
- 产品中心品类/品牌/子品类；
- 供货品类；
- REX 用户分群；
- 渠道或活动分类；
- 报表自定义分组。

这些分类不能因为名称相似就合并。尤其要将“卖什么”的产品分类与“谁供货”的供货品类、以及“谁执行”的销售分群分开。

产品书的职责是解释产品分类本身。供货品类与路径由分销书拥有，人员分群由人员/渠道相关章节拥有。跨书边界应在术语登记中固定。

8. 分类变更的项目检查表

一个产品分类变更至少需要检查：

检查	问题
变更对象	改品类、品牌、子品类，还是新建产品
有效时间	哪个时点开始影响规则和报表
历史	旧分类是否保留、历史报表是否重算
规则	哪些产品范围依赖该分类
报表	分母、分组、筛选和导出是否变化
下游	分销、完美门店、DTR、订单或价格查询是否引用
对账	分类合计与产品总盘是否仍能解释差异
回滚	新分类错误时能否恢复旧版本

不能用一次数据库 UPDATE 代替这套检查。分类更新如果没有版本、触发和下游验收，历史口径会在没有提示的情况下改变。

9. 本章结论与未知

本章可以支持：

1. 产品身份与产品分类回答不同问题；
2. 分类字典范围不等于当前使用范围；
3. 当前 staging 的实际使用值和空值必须显式进入查询设计；
4. 全球品牌组合不能被写成中国当前 SKU 清单；
5. 分类变更需要版本、规则和报表影响分析。

本章未回答：

- 320 行空分类的业务原因；
- 产品中心分类字典的最终业务权威；

- 产品分类与供货品类、REX 分群的完整跨系统映射；
- 分类变更后所有下游是否自动重算；
- staging 与生产的分类分布是否一致。

案头入口与下一章

对象边界和分类字段卡见第 A 卷，分类变更的检查单与未决项见第 C 卷和第 D 卷。下一章从分类语境转向生命周期和有效期，回答“当前产品”为什么必须带查询时点。

第 4 章 产品时间：创建、上市、调整、退市与历史版本

“这个产品当前有效吗”看起来只要求一个状态字段。实际至少有两个问题：该产品在自己的业务生命周期中处于什么阶段，以及这条记录在什么时间窗内适用。

一个产品可能仍在生命周期中，却有一条未来才生效的新包装；一条记录可能在当前时间窗有效，但所属 S2 组合已进入退市状态；一个查询可能没有过滤有效期，于是把历史版本也当成当前。把 `status`、`state`、`is_active` 和 `effective_*` 压成一个布尔值，就会丢失业务含义。

本章的中心问题是：**如何分别解释产品生命周期、记录有效期和历史版本？**

1. 不同产品对象有自己的生命周期

产品中心文档只为 S2 组合产品定义了四个明确生命周期状态：

状态	文档含义
Creation	产品信息录入中，尚未正式发布
Active	已发布，可被业务使用
BillAdjmt	账单调整期，暂停新增订单
Retired	已退市，进入终态

当前代码 `LifecycleStatusEnum` 声明同样的四个值。状态校验允许新 S2 产品从 Creation 或 Active 开始，不允许 Active 回到 Creation，也不允许 Retired 再改成其它状态。

这只是 S2 组合产品的生命周期，不应自动套到 material、昵称、条码或所有 `bz_product` 记录上。不同对象可以有不同状态机和不同有效字段。

2. staging 的 S2 状态不是单一“有效/无效”

2026-10-26 23:05 CST，staging `t2_product_bundle_header_current` 共 24 行：

lifecycle_status	is_active	行数	当前时间窗有效
Creation	true	3	3
Active	true	18	18
BillAdjmt	true	2	2
Retired	true	1	1

最值得注意的是，当前那条 Retired 记录仍然 `is_active=true`，并且仍在当前时间窗内。

这不是矛盾本身，而是两个字段回答不同问题：`lifecycle_status` 表示生命周期阶段，`is_active` 表示当前表的启用标记，`effective_start_at/effective_end_at` 表示记录有效期。三者这次恰好组合成“退市但仍标记 active 且在时间窗内”。

本手册不能根据一个 staging 样本裁定这是历史设计、数据遗留还是错误。项目必须回到 S2 状态机和消费逻辑，确认退市产品的 `is_active` 应如何维护。

3. 通用产品表也有状态和有效期

当前 `bz_product` 表同时有：

- `state` 整型字段；
- `effective_start_timestamp`；
- `effective_end_timestamp`；
- `is_deleted`。

2026–10–26 staging 非删除行按 state 聚合：

state	行数	当前时间窗有效
0	651	651
1	2,464	2,464

本次所有 3,115 条非删除记录都在当前时间窗。这个观察说明“表里有历史行”不能由表名或表结构直接推出，也不能说明 `state=0/1` 的具体业务含义。

产品手册不能把 `state=1` 直接写成 Active。它只在 S2 生命周期中有明确代码枚举；`bz_product.state` 的语义仍需业务字典或代码消费证据。

4. 有效时间与删除标记也不同

`is_deleted` 表示记录是否被逻辑删除。有效期表示记录何时适用。一次更新可能：

- 不删除旧记录，而是关闭旧记录有效期并插入新版本；
- 将旧记录逻辑删除；
- 只修改当前行；
- 生成未来生效行。

当前昵称更新代码展示了第一种：关闭旧 `BzProduct` 记录并插入一条新记录。它说明项目不能只在当前表里找最后更新时间，还必须按业务键和查询时点取正确版本。

一个时间查询至少应写明：

```
business key
+ effective_start_timestamp <= query_time
+ effective_end_timestamp > query_time
+ is_deleted = false
+ object-specific status / active flag
```

`9999-12-31` 常被用作开放结束的哨兵值，但它是存储表达，不是业务日期。查询应说明具体时点，而不是把所有哨兵值当作“永久有效”。

5. 生命周期状态不等于可售状态

即使 S2 产品 Active，也不自动表示：

- 所有渠道都可售；
- 所有客户都已上架；
- 所有门店都进入必铺；
- 价格已经生成；
- 库存可用；
- 经销商产品映射已完成。

可售、价格、规则和库存属于不同链路。产品生命周期只描述产品在本系统中的阶段。

反过来，产品 Retired 也不自动删除历史订单、历史报表和历史组合关系。它的业务意义是当前业务使用需要受限，历史事实仍需可解释。

6. 产品时间影响哪些查询

问题	必须带的时间语义
当前产品叫什么	查询时点 + 昵称有效版本
某 S2 组合当时是否可用	查询时点 + S2 生命周期状态
某历史订单用了哪个组合	订单时间 + 当时组合版本
当前有哪些条码	条码投影时间 + 删除状态
新品何时可进入规则	产品生效时间 + 规则期次
退市后历史报表是否变化	是否重算 + 历史快照 + 报表读取时点

如果这些问题共用一个“最新产品表”，某些问题必然得到不适用的答案。

7. 变更与回归的最小检查

产品状态调整或版本更新前，至少检查：

1. 改的是哪类产品对象，使用哪个状态机；
2. 旧记录是否关闭，新记录何时开始；
3. 历史和未来是否同时存在重叠或空洞；
4. S2 `lifecycle_status`、`is_active` 和有效期是否一致；
5. 下游查询是否按业务时点取版本；
6. 历史订单、报表和规则是否需要重算；
7. 退市是否阻断新使用，同时保留旧事实；
8. 回滚是恢复旧状态还是生成新版本。

8. 本章结论与未知

本章可以支持：

1. S2 组合产品有四个明确生命周期状态；
2. 生命周期状态、启用标记和有效期是不同字段；
3. 当前 staging 存在 Retired 且 `is_active=true` 的记录；
4. `bz_product.state` 的业务语义不能从字段值自行推断；
5. 产品历史查询必须带业务键、有效期与对象特定状态。

本章仍不回答：

- 当前 Retired 记录为何仍 active；
- `bz_product.state=0/1` 的业务字典；
- 所有产品表的历史保留策略；
- 退市对订单、规则和报表的实际阻断范围；
- staging 镜像与本地代码 HEAD 的准确提交映射。

案头入口与下一章

生命周期字段和历史查询提醒见第 A 卷，退市与历史回归检查见第 C 卷。下一章进入组合产品和单位，解释 Header、Line、Unit 以及 TDU、MCU、RSU 如何在同一产品定义中承担不同职责。

第 5 章 组合产品、包装单位与换算

一个组合产品在页面展示、仓库入库、订单下单和报表拆分中可能出现不同粒度。运营看到“套装”，仓库看到“箱”，门店看到“包”，消费者看到“件”。这些粒度如果只用一个数量字段表达，库存、价格和销量就可能在不报错的情况下被混合。

S2 组合进一步增加一层：组合头本身是产品，组合行又引用其它产品。组合头与子项不是同一条记录；包装单位也不等于组合行。

本章的中心问题是：**S2 Header、Line、Unit 和 TDU/MCU/RSU 分别表达什么，数量在哪一层换算?**

1. 三层结构

产品中心文档把 S2 组合产品分为：

层	代表	回答
Header	`t2_product_bundle_header_current`	这是哪个组合产品，生命周期和共享属性是什么
Line	`t2_product_bundle_line_current`	组合中包含哪些子项，每个子项数量是多少
Unit	`t2_product_bundle_unit_current`	这个组合按哪些包装单位出现，如何换算

三层都属于组合产品定义，但职责不同。Header 不是 Line 的汇总字段，Unit 也不是 Line 的单位备注。

2. 组合行用 parent 指向 Header

当前 59 条 S2 行项中，`product_code` 是组合内的子项，`parent_product_code` 才指向组合头。

先用错误连接验证这个边界：

```
line.product_code = header.product_code
```

当前返回 0 个 Header。改用：

```
line.parent_product_code = header.product_code
```

24 个当前 Header 全部关联到行项，共 59 行。

这不是代码细节，而是关系语义。组合维护也要按 `parent_product_code` 查询子项；代码中批量查询行项同样使用 `getParentProductCode`。

3. 每个组合包含多少行项

当前 24 个 S2 Header 的行项数分布：

行项数	Header 数
1	4
2	9
3	9
5	2

组合最少包含一个子项，当前最多观察到五个。样本不能作为业务上限，只能说明行项数量不是固定值。

当前 59 条行项的 `product_bundle_type` 全部为代码注释中的 `0`，即玛氏单品。样本中没有观察到类型 1 的玛氏组合子项，但不能据此断言嵌套组合永远不支持。

4. TDU、MCU、RSU 是包装层级

产品单位文档将三层单位定义为：

层	代码	文档含义
TDU	CS	物流配送单元，通常为箱
MCU	SB	市场销售单元，可选的中间包装
RSU	PCE	零售销售单元，最小销售单位

当前单位配置：

unit_code	基础单位	行数	产品数	conversion_factor
CS	0	24	24	1
PCE	1	24	24	1–48
SB	0	3	3	4–8

24 个组合全部有 CS 和 PCE；3 个还有 SB。对应到单位层数：21 个组合为两层，3 个组合为三层。

当前代码 `validateUnits` 要求 S2 产品必须包含 TDU 和 RSU。TDU 转换系数必须为 1。存在 MCU 时，RSU 不能小于 TDU/MCU，并且必须是 MCU 的整数倍。

5. 换算系数不是一个“箱规文本”

`conversion_factor` 表示 1 个 TDU 等于多少个该单位。以三单位组合为例：

```
1 CS (TDU) = 1
1 SB (MCU) = N
1 PCE (RSU) = M
```

若 `1 CS = 12 SB = 48 PCE`，则 SB 的系数为 12，PCE 的系数为 48。`48 % 12 = 0` 通过整数倍约束；反过来说，不能由 48 个 PCE 自动推出箱内每包有多少件，仍要读取 SB 系数。

`is_base_unit=1` 当前只用于 PCE，表示零售基础单位。它不表示 PCE 就是库存或订单主单位；具体事实粒度仍取决于业务场景。

6. 条码和价格挂在单位

单位表同时有：

- `bar_code`；
- `list_price`；
- 长宽高；
- 分子、分母；
- 转换系数。

这说明同一组合的 CS、SB、PCE 可以有不同条码和价格。把一个条码复制到整个组合，或把箱价直接用于 PCE，会改变业务事实。

价格是否最终用于订单，还要继续看价格链、客户价、价格组和订单换算。产品单位表只提供单位级定义。

7. 数量在哪一层

问题	正确粒度
这个套装包含哪些商品	Line：parent + child + qty
一个套装的箱/包/件规格	Unit：product + unit_code + factor
仓库收到多少箱	事实数量 + CS 单位
门店收到多少件	事实数量 + PCE 单位或换算后数量
报表统计多少单品	组合拆行后 child product + 统一单位
这个组合价格多少	先确认单位，再找对应价格事实

组合定义不会自动决定订单、库存和报表使用哪一层。每一次跨表计算都要声明数量来自哪个单位。

8. 四源案例：从组合头读出三张表

前面的规则只有在同一组合、同一时点和同一单位语境下成立才有业务意义。2026-10-27 的 staging 样本提供了一个可以完整回读的案例：`product_code=90000003`。它不是一个从页面截图中摘出的“看起来像组合”的名称，而是 Header、Line、Unit 三张当前表能够用同一业务键连接，并且本地代码存在对应实体和写入路径。

这个组合当前的生命周期状态是 `Active`，`is_active=true`，有效期从 2025-10-10 开始。Header 显示 TDU、MCU、RSU 分别为“箱”“包”“组”，并分别带有条码。这里先不要急着把三个名称解释成三个产品：它们描述的是同一个 S2 组合在配送、中间销售和零售场景中的三个包装层级。

Unit 表给出了数量关系：

单位层	`unit_code`	当前单位名	转换系数	当前角色
TDU	CS	箱	1	配送基准
MCU	SB	包	8	中间销售单位
RSU	PCE	组	48	当前基础零售单位

因此，本样本的当前配置是 `1 箱 = 8 包 = 48 组`。这个等式来自同一时刻的三条 Unit 记录，不应把 Header 中的三个单位名直接相加，也不能因为 PCE 是基础单位就认为订单或库存必然按 PCE 保存。`conversion_factor` 表示相对 TDU 的数量；它既不是单位自身的主键，也不是所有下游共同采用的事实粒度。

Line 表回答的则是另一个问题：这个组合由什么子项构成。当前三条 Line 均通过 `parent_product_code=90000003` 指向组合，子项的 `product_bundle_type` 都是 `0`，即代码注释中的玛氏单品；但它们的子项编码、单位与数量各不相同。

子项 `product_code`	子项单位	组合内数量
`10249592`	PCE / 包	2
`10249713`	SB / 中盒	1
`10270042`	PCE / 条	5

这三行不能直接相加成“8 个商品”。第一，三行的 `unit_code` 不完全相同；第二，Line 的单位描述属于子项，不等于组合自身的 TDU/MCU/RSU；第三，组合是否拆成订单行、如何换算、是否保留原组合单位，还要由订单和库存实现决定。把 Line 当作包装换算表，会把“组合由什么构成”和“组合按什么单位出售”混成一个问题。

8.1 一次正确的回读顺序

面对类似页面，先以 `product_code` 锁定 Header，确认它当前是否有效、处于什么生命周期。然后分别读取 Unit 和 Line，而不是用一条内连接一次取完。Unit 用来解释该组合的配送、销售和零售包装；Line 用来解释其组成子项。最后再进入订单、库存或报表，确认目标消费者实际采用哪个键。

这个顺序可以避免一种常见误判：从 Header 看到“箱、包、组”，就假定 Line 的三行也分别代表这三个层级。数据库中的 Line 记录并不是包装系数表；它们各自保存子产品的单位与数量。只有把 Header、Unit、Line 分开读取，读者才能判断差异来自组合结构还是包装层级。

8.2 四源证据能够证明什么

本案例把四类证据连接到同一业务键：

证据	当前内容	证明边界
文档	S2 三层结构、单位规则和 API 契约	规定对象、字段与预期关系
代码	三个实体映射及 `createS2Product()` / `validateUnits()`	本地 HEAD 存在三表写入和校验实现
staging 部署	product-nickname 1/1，镜像 `2026-10-13-1136`	服务存在并运行该镜像
staging 数据库	Header 1 行、Line 3 行、Unit 3 行	指定时点可由同一 `product_code` 回读

四源能够证明该组合定义在当前 staging 中自洽存在，也能证明当前单位配置满足 TDU=1、RSU 是 MCU 整数倍的代码校验。它不能证明该组合已经进入订单拆零、库存结批或报表，也不能证明本地代码 HEAD 由当前镜像构建，更不能把 staging 分布外推到生产。

8.3 写入路径告诉项目什么

当前 `createS2Product()` 先完成产品、子项、单位和状态校验，再在事务中写入 Line、Unit 和 Header，最后发送 S2 变更消息。这个实现说明组合不是一个单表保存动作：只修 Header、只补 Unit 或只建 Line，都会留下结构不完整的产品。

但“代码在事务中依次写入并发送事件”不等于“所有消费者已经收到或完成重建”。项目验收仍要分别检查事务结果、消息投递、DI 或缓存刷新，以及订单和报表回读。代码给出可能的传播路径，运行证据才能说明本次传播是否完成。

9. 四个反例

反例一：用 `line.product_code` 连接 Header

它连接的是子项与自身，不是组合头。正确路径是 `line.parent_product_code → header.product_code`。

反例二：只看 Header 数量

Header 数量不是子项数量；包装比例也不在 Header 上。

反例三：把 PCE 基础单位当默认业务单位

PCE 是基础零售单位，不代表订单、库存或报表都按 PCE 存储。

反例四：直接把箱价除以前端显示数量

价格对应单位、税和客户条件可能不同。必须读取单位条码与单位价，再走价格链和订单规则。

10. 本章结论与未知

本章可以支持：

1. S2 由 Header、Line、Unit 三层组成；
2. Line 通过 `parent_product_code` 指向 Header；
3. 当前 24 个组合全部有行项和包装单位；
4. 当前 21 个组合有 TDU+RSU，3 个另有 MCU；
5. 条码、价格与换算系数位于单位层。
6. staging `product_code=90000003` 可由同一键回读 1 个 Header、3 条 Line 和 3 个 Unit。

本章仍不回答：

- 当前组合为何全部只引用 type=0 子项；
- 所有 SB/PCE 换算在业务上是否完整；
- 订单和库存实际使用哪些单位；
- 嵌套 S2 的当前支持范围；
- 组合变更后历史订单和库存的换算回放。
- 该案例是否已经进入订单、库存或报表，以及这些下游是否使用了同一单位口径。

案头入口与下一章

对象和单位卡见第 A 卷，S2 API、数据链与项目用例见第 B 卷和第 C 卷。下一章转入产品主数据的写入与集成，区分业务维护入口、DI 投影和开放查询面。

第 6 章 产品主数据的写入、集成与查询

产品问题进入系统后，第一个回答经常是“去产品中心查”。但产品中心不是一个数据库，也不是一个微服务。它同时有业务写入口、数据集成管道、查询开放面、文件与单位 Provider、实体引擎和前端。

如果项目不先确认写入口和查询面，就可能把 DI 投影当成主数据，把开放 API 当成写入接口，或只凭某个服务 Deployment 在跑就宣布产品主数据完成。

本章的中心问题是：**产品事实从哪里来、在哪里被维护、经过什么集成、最终由谁查询输出？**

1. 当前六个运行面

2026-10-26 staging 产品中心有六个 1/1 Deployment：

Deployment	当前镜像	主要职责
`product-nickname`	`product-nickname:2026-10-13-1136`	产品、昵称、分类、属性、S2 与价格链
`product-locust`	`product-locust:2026-06-16-1522`	查询、输出、开放 API 与导出
`product-sequoia`	`product-sequoia:2026-04-40-0948`	DAQ/DTR/DT 数据管道
`product-poplar`	`product-poplar-1.0.0-rc-2023-06-29-1617.1`	实体/过程引擎
`product-efs`	`product-efs:2026-03-24-1007`	文件服务与产品单位
`product-wps`	`product-wps:2026-0901-1812`	产品管理前端

六个服务都就绪，只说明运行面存在。它们的数据职责和完成语义仍不同。

2. `product-nickname` 是主要业务写入口

文档把 `product-nickname` 定义为产品主数据 SoT，负责：

- 产品昵称维护；
- 分类、品牌和属性引用；
- 成品与 S2 组合；
- 编码定义与映射；
- 价格链模块。

代码中 `ProductNikeNameController` 和 `S2ProductController` 分别提供产品昵称与 S2 的业务入口，`ProductNikeNameServiceImpl` 与 `S2ProductServiceImpl` 承载业务逻辑。

这里的“SoT”必须限定到该服务声明的范围。当前 `bz_product*` 与 `t1/t2/t3_*` 双体系边界仍未裁定，不能把“服务是 SoT”扩写成“每张表、每个编码和每个价格事实都已经统一”。

3. `product-sequoia` 是集成面，不是业务维护页

当前 `product-sequoia` 的数据投影：

投影	行数
`di_otr_product_nickname`	15,783
`di_otr_nickname_barcodes`	1,646
`di_otr_product_uom`	24,042
`di_otr_ws_finishgood`	4,605

这些是 DI/OTR 层，不是产品维护输入。项目发现投影缺失时，应继续查上游原始数据、转换过程和刷新水位，而不是直接修改投影。

`product-sequoia` 代码还包含门店导入校验处理器，但文档已确认实际注册生效在门店中心 `store_sequoia`。代码位于哪个仓库不等于能力归属；运行注册和消费方决定实际边界。

4. `product-locust` 是查询与输出面

当前 `product_locust.api_definition` 有 5 条。服务提供：

- 统一实体查询；
- 输出与导出；
- 修订历史查询；
- 动态路由到 SQL 或 MongoDB；
- App/API 配置和访问分配。

它的 `ProductIdQueryController`、`ProductOutputApiController`、`EntityController` 和 `OutPutApiController` 是查询/输出来源，不是产品昵称写入方。

文档明确 `product-locust` 不负责产品昵称 CRUD，也不负责 DI/ETL。把查询成功当写入成功，会让项目忽略真正的产品维护入口。

5. `product-efs` 是单位事实的重要 Provider

文档和当前 service 描述都把产品单位放在 `product-efs` / `product_uom` 侧，同时 DI 产出 `di_otr_product_uom`。

当前 `di_otr_product_uom` 有 24,042 行，`product_efs.product_uom` 在先前聚合中为 24,121 行。两者接近但不等，且表和刷新语义不同。

这与第 5 章形成直接约束：S2 Unit 是组合包装结构，产品 UOM 是单位与换算 Provider。项目不能假设它们由同一张表、同一时点维护。

6. 写入、集成、查询三张职责表

问题	首选入口	不能替代
维护产品昵称	product-nickname 业务服务	product-locust 查询
维护 S2	product-nickname / S2 service	product-sequoia DI 投影
查看产品 DI	product-sequoia 库与过程	产品业务写库
单位与文件	product-efs	S2 Unit 单独回答全部 UOM
查询/导出/开放接口	product-locust	产品主数据写入
前端操作	product-wps	后端事实与部署

项目应先定位“维护、投影、查询、前端”中的层，再做排障和变更。

7. 当前产品数据流

高层链路可写成：

```
ERP / MDP / 文件输入
  → product-sequoia DI
  → product-nickname / product-efs 等业务载体
  → 产品查询与投影
  → 门店分销 / DTR / 完美门店 / 报表 / SuperView
```

这是职责链，不是所有字段的单一路径。昵称、S2、UOM、价格和客户价可能分别进入不同表和刷新机制。

8. 四个反例

反例一：只看六个服务都就绪

这不能证明产品写入、DI 刷新和开放接口已经一致。

反例二：从 DI 表反写业务主数据

DI 是下游投影，直接修改会破坏可重放性和血缘。

反例三：用 product-locust 页面判断写入成功

查询面可能读取缓存、投影或不同数据源。写入必须回到对应业务入口回读。

反例四：把 product-sequoia 里的门店校验代码归给产品中心

实际注册和消费方在 store_sequoia。共享引擎代码不等于产品业务能力。

9. 本章结论与未知

本章可以支持：

1. 产品中心当前有六个不同运行面；
2. product-nickname 是主要业务写入口；
3. product-sequoia 是 DI 管道；
4. product-locust 是查询/开放面；
5. product-efs 是单位/文件 Provider；
6. 服务就绪不能替代跨层完成验收。

本章仍不回答：

- `bz_product*` 与 `t1/t2/t3_*` 的最终 SoT；
- product-poplar/efs/wps 的精确源码提交；
- DI 过程调度与失败重试；
- 所有开放 API 消费方和数据授权；
- 产品主数据到各下游的实际刷新时延。

案头入口与下一章

服务、API、部署和数据链可在第 B 卷按运行面查询，开放问题见第 D 卷。下一章进入价格批次，解释一次文件上传如何经过匹配、派生、结果写入和历史保留。

第 7 章 价格链：从价格文件到可用价格

价格链不是“把 Excel 导入数据库”这么简单。一个以 `batch_id` 标识的批次要经过文件校验、产品匹配、字段派生、输出组装、去重写入和状态更新；同一条结果还会覆盖旧版本并留下历史。

如果只看到最终表有价格，项目无法回答这份价格来自哪个文件、哪次匹配失败、什么时候被新版本覆盖，或为什么批次失败却仍有旧结果可用。

本章的中心问题是：**一份价格文件如何成为可查询、可追溯的价格结果，批次状态和结果状态为什么不能互相替代？**

1. 价格链是产品服务内的 Module

文档把价格链定义为 `product-nickname` 服务内的 pricing module，不是独立 Deployment。它共享父服务的运行、镜像和主库。

输入由 Horde 上传，服务执行 6 步流水线：

```
读取入库
→ 产品主数据匹配
→ 衍生字段计算
→ 46  字段输出组装
→ 7  键去重写入
→ 批次状态更新
```

SuperView 直连结果表查询和导出。

2. 批次、暂存、结果和历史是四层

层	回答
Batch	这次上传是否被接受、处理到哪一步
Staging	本批次读入了哪些原始行
Final	当前去重后的可用价格结果
History	结果覆盖前的历史版本

当前 staging：

载体	行数
`pricing_batch_status`	329
`pricing_staging_data`	4,685
`pricing_final_data`	2,960
`pricing_final_data_history`	3,235
`pricing_pipeline_step_log`	1,092
`pricing_te_mapping`	34

批次多、暂存多、历史多，最终结果少，不能用一个总行数代表价格覆盖。

3. 当前批次状态

2026-10-26 23:24:21 CST：

status	批次数	total_rows	processed_rows	failed_rows
COMPLETED	129	3,972	3,921	51
FAILED	200	707	0	376

当前没有 PROCESSING 批次。

FAILED 批次中的 `total_rows` 与 `failed_rows` 不相等，说明状态表的行数字段也不能用简单的 total − failed 推断所有业务原因。具体错误需要看 `error_details_json`、步骤日志和原始批次。

按操作类型：

操作	接受	拒绝
FULL_REPLACE	39	48
HORDE_IMPORT	86	147
PC_REIMPORT	3	6

失败批次有专门留痕。审计价值来自“失败的输入也可追溯”，而不是只统计成功批次。

4. 46 字段输出与物理表列数

业务规范与输出字典定义 46 个输出字段，其中 6 个 P&L 字段本期预留。

当前 `pricing_final_data` 物理列有 49 个。物理列数大于业务输出字段，通常说明还含主键、批次、审计或技术列。项目不能拿 `information_schema` 的列数与业务字段数量直接比较，也不能把物理列全部暴露给业务。

5. 去重与历史

结果写入采用 7 字段唯一键去重和 last-write-wins。覆盖前版本写入 `pricing_final_data_history`，历史表 append-only。

代码 `OutputUpsertService` 显示：

- 正常流程先写历史，再执行最终结果 upsert；
- 历史写入有 `pricing.final-history.enabled` 配置开关；
- 历史写入失败会记录错误，但最终 upsert 仍继续。

这要求项目在验收中区分“当前结果正确”和“历史可追溯”。若开关关闭或历史写入失败，当前价格仍可能可用，但追溯链不完整。

6. 产品匹配失败不一定让批次失败

BR-PRICE 明确产品匹配失败不阻断批次：产品字段可置空、行仍进入结果、批次可 COMPLETED，并记录 `UNMATCHED` 与错误码。

因此：

- COMPLETED 不等于每行都匹配到产品；
- FAILED 不等于没有写入任何行；
- 结果表有行不等于业务字段完整；
- 诊断信息是批次状态之外的第二层判断。

项目必须把批次状态、行级处理状态和产品匹配诊断分开统计。

7. 当前结果新鲜度

当前 `pricing_final_data` 最新 `updated_at` 为 2026-10-01 06:27:00，而最新批次失败发生在 2026-10-26。

这说明“最近一次批次失败”和“当前可用结果最后更新时间”不是同一事实。消费者仍可能读到 10 月 1 日的结果，不能因为今天有上传记录就认为价格已刷新。

8. 四源案例：一个 batch_id 能追到哪里

价格链最容易出现“页面说完成、业务不敢使用”的情况。下面的 staging 案例把 `batch_id=8f176d55-f595-42a3-ba65-d39d1db68359` 作为唯一追踪键，展示当前实现能够证明到哪一层，以及仍不能证明什么。

该批次创建于 2026-09-28，操作类型为 `HORDE_IMPORT`。批次状态为 `COMPLETED`，输入 36 行、处理 36 行、失败 0 行，版本 11，当前阶段已经清空。`current_stage` 为空本身不是一个错误：终态批次的阶段指针会被清除，真正的处理轨迹要回到步骤日志。

同一个 `batch_id` 在四层数据中分别出现：

层	当前观察	回答的问题
Batch	1 条 `COMPLETED` 状态	这次上传是否结束，统计是多少
Staging	36 行	本批次读入了哪些输入行
Final	36 行	本批次为哪些去重结果贡献了当前值
History	36 行	结果写入前保留了哪些版本

步骤日志另有 6 行，从 `STEP_1_PARSE` 到 `STEP_6_FINALIZE` 全部为 `SUCCESS`。它们与本地代码中的 `processBatch()` 顺序一致：解析并写暂存、批量产品匹配、计算衍生字段、组装输出、执行 final upsert、更新批次终态。当前代码还在第五步之前写 history，因此历史行与最终结果行可以沿同一批次回查。

这个案例比“结果表里有一行价格”更强，因为批次、步骤和三层数据都能用同一键连接；但它仍不等于业务验收。批次级 `failed_rows=0` 只说明当前统计没有失败行，不能自动推出所有产品都已匹配、所有公式都符合业务口径，或 SuperView 页面按目标筛选返回了同一批结果。

8.1 为什么历史行与当前行必须分开看

当前 `OutputUpsertService` 在 final upsert 前写 append-only history；`pricing.final-history.enabled` 是独立开关。历史写入失败会记录指标，但默认不会阻断 final upsert，因此“当前价格可用”与“历史链完整”必须分别检查。

本案例中 final 和 history 都有 36 行，可以支持“该批次当前观察下两层均留下了数据”。它不能证明每一个历史版本都与业务期望一致，也不能证明其它环境的开关同样开启。项目验收应至少固定：批次、输入键、最终键、历史版本时间和查询结果，而不是只比较两张表的总行数。

8.2 从批次定位到消费结果

排障时先以 `batch_id` 固定批次，再查看步骤日志中的失败或耗时异常。随后按目标业务键回到 staging、final 和 history，确认该行是否经历了匹配、计算和写入。最后才进入 SuperView 或其它消费面，核对页面读取的是当前最终值、历史值还是另一个查询视图。

若下游结果缺失，不能立即重跑批次。应先判断问题发生在输入、匹配、派生、组装、去重、历史、最终写入还是查询面。不同层的修复动作不同，错误输入重跑仍会得到相同失败，查询缓存或视图差异也不能靠重传价格文件解决。

9. 价格链验收卡

检查	问题
文件	原始文件、哈希、批次和 trace 是否可查
校验	7 级校验结果和错误明细是否完整
匹配	产品匹配与未匹配诊断
派生	公式、TE% 和精度是否按版本
结果	当前值与历史值是否同时可追溯
状态	Batch、step、row、product match 分别是什么
查询	SuperView 读到的是哪个更新时点
回滚	重推是否生成新批次与新结果版本

10. 本章结论与未知

本章可以支持：

1. 价格链是 product-nickname 内的 module；
2. 批次、暂存、结果和历史分层；
3. 当前所有批次中 COMPLETED 129、FAILED 200；
4. 当前 final 行 2,960，物理列 49，业务输出定义 46；
5. 最近失败批次不代表 final 已同步更新。
6. staging 批次 `8f176d55-f595-42a3-ba65-d39d1db68359` 可由同一 `batch_id` 回读 36 条暂存、36 条最终、36 条历史和 6 个成功步骤。

本章仍不回答：

- 所有失败错误码的业务根因；
- 当前 final 是否已覆盖全部在售客户和产品；
- 生产分支与当前 staging 是否一致；
- 历史写入开关在目标环境的配置；
- 46 个业务字段的逐字段完整签收状态。
- SuperView 是否按目标筛选返回该批次的最终结果，以及 staging 镜像是否由当前本地代码 HEAD 构建。

案头入口与下一章

价格链数据流和证据模板见第 B 卷，价格方案工作台见第 C 卷。下一章转入门店分销与完美门店，观察产品身份怎样进入规则、采集和报表消费。

第 8 章 产品怎样进入门店分销与完美门店

产品中心不是说“有这个产品”，门店分销就能自动回答“这家店应该铺什么”。规则需要产品范围，完美门店需要 SKU 级采集，报表需要 material 或条码，门店侧还要把这些键映射回同一个业务产品。

产品进入下游的难点不是一次调用，而是不同消费方使用不同键：

- 分销规则以 nickname 为主；
- 门店报表可直接读成品条码和 material；
- 完美门店插件以 materialcode 为核心；
- 部分消费方调用 product-nickname API，部分直接读 DI 表。

本章的中心问题是：**产品主数据怎样成为门店分销与完美门店的共同语言，哪些差异必须显式对账？**

1. 分销规则当前使用 217 个昵称

当前两条生效规则含 217 个 distinct nickname。

与产品中心按精确昵称文本比较：

目标载体	命中
`product_sequoia.di_otr_product_nickname`	217 / 217
当前有效 `product_nickname.bz_product`	143 / 217

这说明：

- 当前规则昵称在 DI 投影中完全可按名称找到；
- 只有 143 个能在当前产品业务表按同名文本找到。

但这不是一个可直接判定的“74 个产品缺失”：

- 比较的是名称文本，不是稳定 `nickname_code`；
- DI 投影没有 `nickname_code` 列；
- 业务表可能存在名称展示差异、历史状态或不同范围。

正确行动是补产品稳定键映射，而不是按名称直接补数据。

2. 两种消费通道同时存在

2.1 API 消费

`fdw-fns` 通过 `ProductNicknameApi` 调用：

```
/product/nickname/all
```

`re-store-sdrs` 也调用 `/product/nickname/all` 和按 code 批量查询。

API 消费让规则能即时取产品候选，但调用时需要应用授权、网络和返回版本。项目不能只看调用成功，还要检查返回是否完整、是否过期。

2.2 直接数据库投影

`re-store-report-generator` 直接读取：

```
product_sequoia.di_otr_ws_finishgood
```

获取 material 与 MCU/RSU 条码；完美门店插件也大量使用 `materialcode`。

直接读取减少 API 跳转，却让产品刷新时点与报表运行强耦合。查询成功不代表产品投影已刷新。

3. 分销规则的产品身份

规则的应分销表以 nickname 为核心，但产品业务身份可能是 material/zrep。一个规则名称要进入采集和报表，必须继续映射到：

- SKU 收集项；
- material/barcode；
- 产品组；
- 完美门店目标字段；
- 实分销来源结果。

任何一步缺少稳定键，可能出现“规则里有、报表里对不上”或“同名称两个产品合并”。

3.1 一个“新产品”要求的名称回连案例

2026-10-27 的 staging 提供了一个典型但不完整的跨系统样本。已生效规则 `345b4f3a-b24f-4a2b-9645-0ac43c2ee2e3` 包含 390 行 `rule_type=新产品` 的产品要求，共 33 个 distinct `nickname_code`。这说明规则侧已经保存了“这条新品要求对应哪个昵

称”的业务范围。

取其中一个要求，`nickname_code=10202`，名称为“绿箭桂花乌龙薄荷味口香糖5片装13.5g”：

数据面	观察结果	当前能回答
分销规则明细	11 行，保留 `nickname_code=10202`	规则要求什么昵称
`product_sequoia.di_otr_product_nickname`	精确名称命中 2 行，`zrep_code=611153`	投影中存在同名产品
`product_nickname.bz_product`	按 `nickname_code=10202` 为 0 行	以该编码直接回连失败
`product_nickname.bz_product`	按精确名称为 0 行	名称回连也未命中当前业务表

这不是一个可以立刻宣布“74 个产品缺失”的证据。DI 投影当前没有 `nickname_code` 列，规则只能先按名称文本连接；而名称相同不代表稳定身份相同，`zrep_code=611153` 也不能仅凭一次同名命中就被认定为规则要求的最终产品键。产品业务表未命中还可能是投影范围、维护时点或当前记录状态造成的，但这些原因本身仍需证据。

对该案例正确的判断是：新品规则要求可以沿名称找到投影候选，但稳定 ID 链尚未闭合；规则缓存是否已生成、门店是否收到该要求、订单和报表是否使用同一产品，也都还没有证明。项目下一步应补 `nickname_code` 到 DI 的稳定映射与缓存 trace，而不是用名称命中直接改写规则或补产品数据。

4. 完美门店怎样使用产品

完美门店是标准、表单、采集、明细、算分和截P的综合结果。产品在其中承担两类角色：

角色	用途
产品维度	按 SKU/品类/产品组计算目标与实际
采集键	将手填、IR 或 EO 结果对到目标 SKU

产品中心提供 material、昵称、条码和组合；门店侧根据渠道制品和报表字段选择具体键。

5. 当前明细与截P证据

2026-10-26：

表	行数	distinct 门店	拜访日期	最新修改
`zo_bysku_detail_p`	3,028,826	1,107,559	2025-12-24 至 2026-10-26	2026-10-26
`zo_ps_all_detail_p`	4,797,030	1,101,315	2026-01-13 至 2026-09-22	2026-09-23

两张表的范围和最新日期不同。bySKU 最新到 2026-10-26，全店型截P最后拜访日期停在 2026-09-22。

这不是自动判定某个产品缺失。必须先确认：

- 两表统计目标是否相同；
- 哪些渠道进入哪张表；
- 最新修改是否只是历史修复；
- 表的刷新水线；
- 目标期次。

6. 产品变更到门店影响

产品变更	门店侧检查
新昵称	规则产品范围、SKU 收集项、报表名称
新条码	扫描、IR、报表条码映射
换 material	完美门店插件、订单/库存、历史报表
UOM 变化	数量、箱规、报表聚合
组合变化	订单拆分、门店目标、采集 SKU
分类变化	品牌/品类规则、报表分组

“产品中心改了字段”只是影响链起点。必须沿消费方逐层回读。

7. 一个跨系统产品对账包

至少包含：

证据	内容
产品业务表	material/zrep/nickname/UOM 当前与历史
DI 投影	产品、条码和 UOM 行及刷新时间
分销规则	nickname、产品组、期次与规则状态
采集	手填/IR/EO 对应的产品键
完美门店	目标、实际、算分和截P
代码	API 或直读路径、消费版本
运行	部署镜像、数据库时点和报表水线

8. 本章结论与未知

本章可以支持：

1. 当前分销规则含 217 个昵称；
2. 217 个昵称按文本全部在 DI 投影中，143 个在业务表中；
3. 产品消费同时使用 API 和直接 DB 投影；
4. bySKU 与全店型截P处于不同时间和范围；
5. 产品变更必须按下游键逐层对账；
6. 已生效规则中的“新产品”要求可在 DI 中按名称找到候选，但尚缺 `nickname_code` 稳定映射。

本章仍不回答：

- 74 个业务表未按同名文本命中的真实映射；
- API 与 DI 投影的刷新时差；
- 各渠道制品实际使用的产品键；
- 全店型截P比 bySKU 更早的原因；
- 生产环境的同类覆盖差异。
- 规则 `nickname_code=10202` 到 DI `zrep_code=611153` 的稳定身份映射，以及该要求是否进入门店缓存和执行结果。

案头入口与下一章

产品消费路径见第 B 卷，跨系统对账和项目检查见第 C 卷。下一章沿经销商产品映射、订单、库存和报表继续向下游追踪，判断产品身份在交易链中怎样被保持或转换。

第 9 章 产品怎样进入经销商、订单、库存与报表

产品进入 DTR 后，往往不再只有 material 或 nickname 一个键。经销商会建立自己的产品主数据、组合和拆零产品，订单行还带交易单位；库存按经销商产品和仓库持有，报表再按 Mars SKU 或条码聚合。如果产品中心、DTR 映射、订单行和报表各自被当成“最终产品事实”，团队就会在不同系统里寻找同一 SKU，却得到不同答案。

本章的中心问题是：**一个 Mars 产品怎样经过经销商映射、订单、库存和报表，仍能被追溯回同一业务身份？**

1. 产品链在 DTR 中至少经过四层

层	主要载体	回答
Mars 产品	`product_nickname` / product DI	Mars 产品是谁
经销商产品与映射	DTR `t6` / `t7` / `t1_wb_dtr_product_mapping_*`	经销商系统里的产品对应谁
订单行	`doc_runtime_staging.t1..t5`	本次交易用了哪个产品与单位
库存与报表	`t11_*` / `zo_*`	当前库存或用哪个 SKU/条码呈现

这些层由不同服务维护，不能靠一次产品中心查询全部回答。

2. 当前 DTR 业务表规模

2026-10-26 DTR staging：

业务面	行数
产品上下架	7,180
可售白名单	16,175
价格组 Header / Line	214 / 186,979
实时库存 current	498,371
库存变动	14,844

这些数不是“产品数”。一个产品可能没有上下架、价格和库存行；一条库存行也可能按经销商产品与仓库重复。

3. 产品映射不是可售和库存

DTR 产品映射回答“经销商产品对应哪个 Mars 产品”。白名单回答“经销商能否卖”，上下架回答“当前状态”，价格组回答“用哪套价格”，库存回答“数量”。

同一 Mars 产品可能：

- 有映射但未在白名单；
- 在白名单但未上架；
- 已上架但无库存；
- 有库存但门店范围不可售；
- 可售但订单单位映射不完整。

因此需要一条完整状态链，而非一个产品主表。

4. 订单行如何使用产品

订单 E2E 说明：

- EO/ERP 订单先校验门店、仓库、产品与规格；
- ERP 订单可先走 Dummy 门店；
- P 价/GSV 与优惠计算依赖产品规格和主数据；
- 二包计算读取组合关系并拆玛氏单品；
- 订单派生层继续产生 SKU 明细。

产品链停摆可能表现为订单 WARN：缺产品数据、单位匹配失败或未匹配订单。这些不是单纯订单代码问题。

5. 库存的“产品”可能是经销商产品

DTR 实时库存和库存变动来自经销商/DMS，产品键可能先落在经销商产品，再经映射解释为 Mars 产品。因此查询“Mars 产品库存”时，必须先确认：

- 库存源是否使用经销商产品编码；
- 映射是否当前有效；
- 仓库/经销商范围；
- UOM 与单位换算；
- 库存新鲜度。

当前 `inventory_live_current` 有 498,371 行，但这些行不能直接回答“有多少 Mars SKU 有库存”。库存事实可能按经销商产品与仓库重复，产品映射和重复粒度尚未完成对账。

6. 报表的键可能回到 material 或条码

`re-store-report-generator` 直接读：

```
product_sequoia.di_otr_ws_finishgood
```

并使用 material 与 MCU/RSU 条码。完美门店插件也常用 `materialcode`。

这意味着订单中的经销商产品、库存中的经销商键，最终必须能通过映射链回到 material 或条码，才能进入 SKU 报表。否则，报表虽然生成成功，也无法证明它统计的是同一组 Mars 产品。

7. 当前产品昵称覆盖差异

分销规则的 217 个 nickname 在 DI 投影中全部按名称命中，但当前 `bz_product` 只命中 143 个。

这个差异提醒项目：

- 当前规则链可能更依赖 DI 投影；
- 业务主表与 DI 的表名和范围不同；
- 名称文本不是稳定对账键；
- 需要 `nickname_code`、material、zrep 和经销商产品键共同连接。

8. 产品跨系统对账顺序

```
Mars material / zrep / nickname
  → product DI
  → DTR Mars product out
  → 经销商 product mapping
  → 可售/上下架/价格
  → 订单行 + UOM
  → 库存行
  → 报表 material / barcode
```

每一步都保留来源键和转换结果。最终报表值不能直接替代中间映射证据。

9. 本章结论与未知

本章可以支持：

1. 产品需经过 Mars 产品、经销商映射、订单和库存/报表多层；
2. DTR 当前有独立可售、上下架、价格组和库存数据；
3. 订单和 P 价依赖产品规格；
4. 报表最终可回到 material/条码；

5. 产品名称覆盖差异要求稳定键对账。

本章仍不回答：

- Mars 产品到经销商产品的完整映射率；
- 订单单位到 Mars UOM 的全量一致性；
- 库存产品键的当前映射覆盖率；
- 产品链停摆对订单失败率的量化影响；
- 生产环境与 staging 的差异。

案头入口与下一章

DTR、订单、库存与报表消费链见第 B 卷，项目验收与对账模板见第 C 卷。下一章把前面的身份、单位、时点和投影放在同一条对账链上，解释数据一致与差异裁决。

第 10 章 产品一致性：同名、多码、单位、时点与投影

两个页面都显示“同一产品”，却给出不同的编码、条码、单位或状态；两张表都叫产品投影，却包含不同行数。项目最自然的反应是要求一个“正确数字”，再让对方系统改到一致。

但“一致”首先要求两件事说的是同一个事实。产品名称、material、zrep、nickname、包装单位、时间窗口和投影用途任意一项不同，数字就可能本来不可比。先减总数或按名称连接，只会把语义差异包装成数据错误。

本章的中心问题是：**怎样证明两条产品记录讲的是同一个对象、同一个单位、同一个时点，而不是仅仅拥有相似的名称或编码？**

1. 一致性不是“所有表相同”

产品链中的不同表面承担不同职责：

表面	主要回答	天然可能不同的原因
`bz_product`	产品、material、zrep、昵称的组合业务视图	维护范围、昵称缺失、历史版本
`bz_ws_finished_good`	成品物料及包装条码	成品维护范围、条码完整度
product DI	面向下游的数据集成投影	同步范围、字段裁剪、刷新时点
DTR 产品映射	经销商产品对应哪个 Mars 产品	经销商主数据与映射时序
S2 Header / Line / Unit	组合、结构和换算	业务对象与层级不同
规则 / 报表	以 nickname、material 或产品组执行业务	聚合粒度与用途不同

这些表面应能在明确键、单位和时点下连接，不应被要求所有行数相等。把“可追溯”误写成“完全相同”，会诱导团队删除合法的一对多、空值和适用范围。

2. 同名的风险

产品名称是给人看的业务语言，不是稳定身份。当前代码与 staging 已表明：

- `nickname_code` 可以为空；
- 一个 nickname 可以关联多个 material；
- 一个 zrep 可以关联多个 material；
- 条码字符串可能重复，必须结合包装单位解释。

因此，按名称比较只能用于生成调查线索，不能作为最终对账键。尤其在分销规则与产品投影之间，名称命中率高，不代表材料、包装和状态都一致。

一个安全的名称比较要同时记录：

```
归一化规则
+ 比较范围
+ 观察时点
+ 名称来源
+ 后续稳定键追查
```

当前 217 个生效分销规则 nickname 在 DI 投影中按名称全部命中，而当前 `bz_product` 只按名称命中 143 个。这个差异可以证明“不同表面使用了不同范围或维护方式”，不能直接推出“74 个产品不存在”。

3. 多码不是自动异常

当前 `bz_product` 的 3,115 条记录对应：

- 3,115 个 distinct material；
- 1,355 个 distinct zrep；
- 1,247 个 distinct nickname；
- 320 条空 `nickname_code`。

这些基数说明同一张当前产品表中已经存在多对一、空值和非全量覆盖。它们是否合理，要看字段语义：

- material 更接近物料；
- zrep 带物料单位语境；
- nickname 是业务引用和展示；
- barcode 是包装或扫描载体；
- S2 产品则是组合结构。

“一个业务商品有多个码”可能来自不同包装、不同系统键或不同聚合对象。真正要拒绝的是在未声明粒度时假设一对一。

4. 当前 material 对账揭示了什么

2026-10-27 06:41—06:43 CST，对当前有效的 `bz_product` 与 `bz_ws_finished_good` 按 distinct `material_code` 做双向检查：

方向	当前位置	对侧缺失
产品 → 当前成品	3,115	10
当前成品 → 产品	3,105	0

所以当前快照中有：

- 3,105 个产品 material 能匹配当前成品；
- 10 个产品 material 没有当前成品匹配；
- 所有当前成品 material 都能匹配当前产品。

本轮材料可以支持“当前存在 10 个单向未匹配 material”。它不能支持：

- 这 10 个产品已经退市；
- 这 10 个产品是脏数据；
- 成品表应补齐 10 行；
- 产品表与成品表应该一一对应；
- staging 与生产具有相同分布。

缺失可能来自成品维护范围、生命周期、同步时点或两侧对象定义。没有业务 Owner 的裁定和消费证据，不能自行选择修复方向。

5. 单位差异必须进入连接条件

同一 material 在不同包装层级下可能具有不同的 TDU、MCU、RSU 和换算系数。连接订单、库存和报表时，至少要带上：

```
material
+ zrep / UOM
+ trade unit
+ conversion context
+ effective time
```

如果一侧是一箱、另一侧是一包，直接按数量相加会得到“数值一致但业务不一致”的结果。当前 S2 校验要求 TDU、RSU 存在，并约束换算关系；这只能证明 S2 写入校验，不应扩写为所有历史订单和库存单位都已等价。

单位对账还应区分：

差异	需要先问
数量相差整数倍	包装换算是否正确
一侧为空	该单位是否为必填、投影是否裁剪
同一 material 多条	是否为不同 zrep/UOM
订单与库存不同	交易单位与库存结存单位是否相同
报表为小数	是否已按换算系数归一

6. 时点差异不是数据漂移

产品链条至少包含四类时间：

1. 产品对象和记录有效期；
2. 昵称、成品和条码的维护时间；
3. DI、DTR 和规则缓存的刷新时间；
4. 订单、库存和报表的事实时间。

当前两张完美门店截P的最新拜访日期分别为 2026-10-26 与 2026-09-22。它们粒度、门店集合和刷新路径不同，不能用总行数差解释产品覆盖。

比较产品数据前，应固定同一观察时点：

```
query_time
+ effective window
+ source last_update
+ target refresh watermark
```

如果一侧是今天的产品主数据，另一侧是昨天生成的 DI，差异可能只是投影尚未刷新。反过来，如果目标时点相同而记录长期不一致，才进入键或规则调查。

7. 投影差异不是源表副本

DI、DTR 和报表投影通常只保留目标消费所需字段，并可能改变粒度。当前已知：

- product DI 的昵称条码投影没有完整 `nickname_code`；
- DTR 产品链有 Mars 产品、经销商产品映射、订单行和库存多份结构；
- 门店报表直接读 `product_sequoia.di_otr_ws_finishgood`；
- 完美门店明细以 material 或 nickname 参与计算。

因此不能要求“源表每一列都在投影中出现”。应要求：

1. 投影声明来源对象和过滤规则；
2. 关键业务键与时间字段可追溯；
3. 投影缺失时能回到源对象；

4. 下游计算所需单位已明确；
5. 刷新水线与业务承诺一致。

“源表存在”只证明来源候选，“投影存在”只证明目标表面。只有在具体消费目的下能够“下游命中”，才能证明该消费链闭合。

8. 五层对账方法

建议按以下顺序处理产品差异：

第一层：先判对象

两边比较的是 material、包装、昵称、S2 组合、经销商产品，还是报表产品组？

第二层：声明粒度

一行代表物料、门店×SKU、订单行、库存批次，还是期次快照？

第三层：选择稳定键

优先使用 material、nickname_code、zrep + UOM 或目标系统明确声明的映射键；名称只做辅助。

第四层：对齐时间

两边都过滤删除标记、状态和同一业务时点，并记录各自最后更新时间。

第五层：解释剩余差异

逐条区分：

- 合法的一对多；
- 空值与不适用；
- 历史、当前和未来；
- 投影未刷新；
- 稳定键缺失；
- 真实维护异常。

没有完成前三层，剩余差异不能进入“修数清单”。

9. 一致性验收矩阵

编号	场景	必须断言
PC-01	同名不同 material	名称只生成线索，稳定键能解释
PC-02	同 material 多 zrep	包装单位与用途正确
PC-03	nickname 为空	查询明确过滤或处理，不静默丢失
PC-04	同一 zrep 多 material	聚合粒度和分配规则正确
PC-05	条码重复	包装层级和扫描事件可解释
PC-06	产品与成品单向缺失	记录方向和范围，不自动补行
PC-07	订单与库存单位不同	换算后金额/数量可复算
PC-08	当前与历史并存	两端使用同一业务时点
PC-09	DI 未刷新	不误判为源产品删除
PC-10	报表投影裁剪	所需键、单位、时间和来源仍可追溯
PC-11	规则 nickname 命中但 material 不同	不把名称命中写成产品一致
PC-12	DTR 映射缺失	订单、可售、价格和库存分别判定

每个用例必须给出环境、观察时间、源表、目标表、键和查询口径。

10. 三个反例

反例一：先减总行数

`bz_product` 3,115 行、成品表 3,105 行，差 10 不能直接推出“多 10 个产品”。只有按 distinct material 双向检查后，才能说存在 10 个产品→成品未匹配。

反例二：按名称强制一对一

名称匹配成功，仍可能一名称对应多 material；名称不匹配，也可能只是 nickname 维护差异。名称不能承担唯一键。

反例三：用今天的维度重算历史

产品后来换包装或条码，不能覆盖历史订单当时的单位。历史报表需要保留当时键和版本，或将可接受的追溯规则写成显式口径。

11. 本章结论与未知

本章可以支持：

1. 产品一致性应检查键、粒度、单位、时点和投影用途；
2. 当前产品与成品不是按总数直接一致；
3. 2026-10-27 staging 观察到 10 个当前产品 material 无当前成品匹配，反向为 0；
4. 名称匹配只能作为调查线索；
5. 投影差异必须回到来源、过滤、刷新和消费目的解释。

本章仍不回答：

- 10 个单向缺失 material 的业务原因；
- product DI 与当前业务表的最终 SoT；
- Mars 产品到经销商产品、订单单位和库存单位的完整覆盖率；
- 所有条码重复和 nickname 共用的合法性；
- 任一缺失是否已影响具体规则、订单或报表；
- 生产环境的产品分布与 staging 是否一致。

案头入口与下一章

对象、字段和时间卡见第 A 卷，项目对账与证据包见第 C 卷，未决差异见第 D 卷。下一章进入变更影响与回归，把产品稳定键传播到规则、执行、交易、报表和历史。

第 11 章 产品变更影响分析与回归

改一个昵称像是文案工作，换一个条码像是补一个字段，调一个 UOM 像是修换算；可一旦这些变化进入规则、订单、库存和报表，结果可能从“页面显示不同”升级为“目标 SKU 错、数量放大、历史无法复算”。

产品变更的风险不在字段数量，而在身份传播。若项目只修改输入页并做一次页面回查，却没有沿稳定键验证 DI、规则缓存、门店采集、DTR 映射、订单和报表，就无法判断变更结束。

本章的中心问题是：**一次昵称、条码、包装、UOM、组合或生命周期变更会影响哪些事实，回归应该按什么键和时点组织？**

1. 先判断变的是哪类对象

变更	主对象	最先要保留的键
昵称或业务显示名	nickname	nickname_code、material、旧新名称
条码新增或替换	包装/扫描载体	material、barcode、包装层级
新增包装	UOM / 成品版本	material、zrep、TDU/MCU/RSU
UOM 换算	单位关系	material + trade unit + conversion
S2 组合结构	组合 Header / Line / Unit	S2 product code、line、unit
生命周期调整	单品、组合或记录版本	业务键、状态、effective window

如果变更单只写“更新产品 X”，影响分析还没有起步。必须先说明对象、包装、单位和生效时点。

2. 名称变更不是只改显示

名称可能被页面读取，也可能被规则匹配、异常诊断、文件名或人工对账使用。当前证据已经表明：

- 规则使用 nickname；
- DI、业务表和报表的名称范围不同；
- 217 个当前规则 nickname 在 DI 中按名称全部命中，而 `bz_product` 只命中 143 个；
- 名称不是稳定对账键。

因此名称变更至少检查：

```
nickname_code
  → 旧/新 nickname
  → 产品业务表
  → product DI
  → 规则产品范围与缓存
  → 门店表单显示
  → 报表列与下游检索
```

若旧名称存在于外部文件、规则或用户收藏，还要确认是否保留别名或迁移映射。直接覆盖名称可能让历史材料无法解释。

3. 条码和包装变更要沿扫描层级回归

条码可能属于 TDU、MCU、RSU 或其它扫描载体。新增条码时只补一个字符串不够，还要确认：

1. 它属于哪个 material 和单位；
2. 是替代旧条码、并行存在还是历史恢复；
3. 目标设备是否读取该层；
4. 重复条码是否已有业务范围；
5. 历史扫描事件是否仍可回查。

当前成品表有 208 行缺少 TDU 条码、320 行缺少 RSU 条码，条码投影也存在重复字符串。项目不能把“有 material”当成“所有扫描层已完整”，也不能在没有业务解释时批量补成同一格式。

4. UOM 变更会放大或缩小业务数值

UOM 回归不能只看单位字段。应选择一条实际订单或库存事实，用以下链复算：

```
material + zrep + trade unit
  → 包装换算
  → 订单行数量/金额
  → 库存结存
```

```
→ SKU 明细
→ 报表聚合
```

至少构造：

- TDU 到 MCU；
- TDU 到 RSU；
- MCU/RSU 不整除；
- 同一 material 多 UOM；
- 历史订单使用旧 UOM；
- 新订单使用新 UOM；
- 报表同时汇总旧新交易。

“新订单正确”不证明历史报表仍可复算；“换算后总量相等”也不证明行级单位都正确。

5. S2 组合变更要区分三层

S2 组合至少包含 Header、Line、Unit。新增、替换或删除组合单品时，应逐层验证：

层	回归问题
Header	组合产品身份、生命周期和有效期
Line	包含哪些单品、数量与生效版本
Unit	组合销售/扫描/配送单位如何表达

然后继续验证组合能否拆到订单单品、门店采集能否识别、报表是否能按预期聚合。组合展示正确不等于订单可拆，订单可拆也不等于报表不重复。

6. 生命周期变更不是删除

产品退市或暂停使用至少涉及：

- 生命周期状态；
- 记录有效期；
- DI 当前投影；
- 规则是否继续要求；
- 门店是否仍显示采集项；
- DTR 是否仍可售；
- 新产品是否允许下单；
- 历史订单、库存和报表是否保留；
- 价格是否仍有未来版本。

当前 S2 曾观察到 Retired 且 `is_active=true`、仍在记录时间窗内的组合。这个观察说明状态、启用标记和有效期必须分别解释，不能用一个 `active` 布尔值覆盖生命周期。

7. 产品变更影响图

建议每条产品变更保留以下影响图：

```
产品源对象
  ├─ DI / 开放查询
  ├─ 分销规则与缓存
  ├─ 门店拜访与完美门店
  ├─ DTR 产品映射/白名单/价格/库存
  ├─ 订单行与单位
```

```
├─ SKU 明细
└─ 截P/报表/下游看板
```

每条边要写：

项	必须记录
连接键	material、nickname_code、zrep/UOM、DTR product key
过滤	删除、状态、有效期、渠道
版本	规则、DI、制品、镜像
时点	源更新时间、投影水线、业务期次
证据	查询、日志、API 返回或运行产物

没有边的系统不自动“无影响”；可能是消费方尚未识别。影响图必须区分“已证实无依赖”和“未发现证据”。

8. 回归分层

8.1 身份层

核对 material、zrep、nickname、barcode、UOM 的旧值、新值和基数。

8.2 投影层

核对 DI、综合查询和目标投影是否包含新版本，旧版本是否按策略关闭。

8.3 规则层

用目标门店样本验证规则命中、产品范围、产品组和缓存重建。

8.4 执行层

验证拜访表单、手填/IR/EO 配置和实际结果能识别同一产品。

8.5 交易层

验证 DTR 映射、上下架、价格、库存、订单单位和行级计算。

8.6 报表层

以同店、同产品、同单位、同期次复算明细、分销分和截P。

8.7 历史层

选旧变更前样本，证明历史记录仍按当时版本解释。

9. 回归矩阵

编号	变更	必测
PCI–01	昵称替换	旧新 nickname_code、规则与 DI 检索
PCI–02	条码新增	包装层级、扫描、识别与去重
PCI–03	条码停用	新扫描阻断、历史扫描保留
PCI–04	新增 UOM	订单、库存、报表换算
PCI–05	UOM 系数修改	旧新订单与历史报表
PCI–06	新产品进入规则	目标、缓存、拜访采集与分数
PCI–07	S2 单品替换	Header/Line/Unit、订单拆分与报表
PCI–08	产品退市	新使用阻断、历史保留
PCI–09	DI 重跑	当前投影、历史时点与下游读值
PCI–10	价格版本	batch_id、匹配、46 字段与 SuperView
PCI–11	经销商产品映射	白名单、价格、库存与下单
PCI–12	回滚	源、投影、规则、价格与报表成套恢复

10. 反例

反例一：按名称全局替换

名称相同或相似的产品可能对应不同 material。文本替换会把其他产品一并改掉。

反例二：只验产品中心页面

页面能查新值，只证明源查询。DI、规则和报表可能仍读旧版本。

反例三：只验一张报表

Bysku、渠道明细、截P和全店型表粒度和时点不同。单表通过不能替代整个报表族。

反例四：退市后物理删除

删除会破坏历史订单和报表解释。生命周期调整不等于历史事实消失。

11. 本章结论与未知

本章可以支持：

1. 产品变更影响应按对象、稳定键、单位和时点展开；
2. 昵称、条码、UOM、S2 和生命周期的回归路径不同；
3. 名称和总行数不足以判断产品一致；
4. 产品变更必须同时验证当前消费和历史解释；
5. 未发现消费证据不等于已证明无依赖。

本章仍不回答：

- 任一真实产品变更从源表到全下游的完整四源案例；
- 两套产品体系的最终写入口；
- 产品到经销商产品映射的完整覆盖率；
- 所有报表对旧新 UOM 的历史重算策略；
- 各外部文件、收藏或人工流程是否保存旧名称；
- staging 与生产变更时延是否一致。

案头入口与下一章

变更工作台和证据包见第 C 卷，仍未闭环的 SoT、UOM 和生产状态见第 D 卷。下一章进入项目交付、运维与演进，说明怎样把影响分析、回归和证据组织成可验收的项目。

第 12 章 产品项目、运维与演进

产品项目结束时最常见的虚假完成信号是“接口返回成功”“页面已经能查”“Pod 正在运行”“代码分支已合并”。这些分别证明一段实现存在，却都不能单独证明产品已正确进入规则、订单、库存、报表和历史。

产品运维也不是收到异常后再查一张表。它需要先知道每类数据由谁写、按哪个键、何时刷新、哪套制品正在运行，以及当源、投影和目标结果冲突时先查哪一层。

本章的中心问题是：**新品、换码、包装/UOM、S2、价格和数据修复项目怎样交付，之后怎样保持可追溯与可演进？**

1. 先分项目类型

类型	典型目标	主要交付面
新品引入	新 material 从上游进入可执行链	主数据、DI、规则、门店、DTR、报表
换码/换包装	新旧身份并存或替换	material/zrep/barcode/UOM、历史兼容
UOM 调整	修正或新增换算	包装、订单、库存、报表
S2 组合	新建或调整组合结构	Header/Line/Unit、订单拆分与消费
价格方案	上传新价格并形成版本	batch、匹配、派生、final/history、SuperView
SoT/模型调整	确定新写入口或统一产品对象	代码、DDL、迁移、双跑、回滚
数据修复	修某批不一致	根因、范围、幂等、历史影响
退市/停用	阻断新使用、保留历史	生命周期、规则、可售、订单、报表

一个项目可能同时包含多类。先拆成可独立验收的变更包，避免“产品主数据升级”变成无法证明范围的巨型项目。

2. 项目前置事实卡

进入设计前至少写清：

1. 变更是产品、包装、昵称、单位、组合、价格还是投影；
2. 业务键和新旧值；
3. 目标环境与现行版本；
4. 生效、停用和历史时点；
5. 受影响渠道、客户、门店和报表；
6. 业务、数据、应用、测试和运维 Owner；
7. 正例、空值、历史和回滚样本；
8. 完成定义与阻断条件。

若当前目标环境中的规则、DI、制品或表结构尚未取证，项目尚未具备声称“可实施”的完整基线，只能进入发现阶段。

3. G0—G7 交付路径

产品项目按八个连续门推进，但每一门解决的事实问题不同。G0 先确定产品对象、业务键、历史和验收目的，不能只写产品名称；后续各门依次固定权威版本、数据与生命周期、影响面、实施方式、回归范围、上线内容和值守收口。前门没有证据时，后门不能靠“代码已完成”或“环境已运行”越级通过。

阶段	主要动作	必须形成的证据
G0：范围与业务键	确认产品对象、业务键、历史和验收目的	范围卡、业务键、正反例
G1：权威与版本	固定文档、代码、部署、任务和表结构基线	版本与运行坐标
G2：数据与生命周期	确认新旧产品、包装、单位和允许空值	修改前后数据与 Owner 确认
G3：影响分析	沿第 11 章影响图识别下游	规则、门店、DTR、订单、库存、报表清单
G4：实施与迁移	通过批准写入口修改，DDL 先行	脚本、迁移、幂等记录
G5：回归	按稳定键验证当前消费与历史解释	七层回归结果
G6：上线	发布代码、配置、DDL、DI/规则和价格版本	发布包、回滚点、监控
G7：值守与收口	观察首个业务周期并核对下游	指标、缺陷、收口结论

G1：权威与版本

建立基线：

```
文档版本
+ 代码仓/分支/提交
+ Deployment 镜像/profile
+ DI/场景/任务版本
+ 当前表结构与数据观察时间
```

4. 交付证据包

层	最小证据
需求	范围、规则、业务 Owner、验收定义
代码	仓库、提交、文件/符号
运行	Deployment、镜像、profile、任务/场景
配置	产品、S2、UOM、规则、价格参数
数据	修改前后快照、batch_id、行数与键
测试	正负样本、历史、重跑、回滚
下游	规则缓存、门店查询、DTR、订单、报表回读
运维	日志、监控、告警、保留和恢复

证据必须通过同一业务键和时间窗口连接。零散截图不能补上缺失的跨源关系。

5. 价格链是嵌入式模块，不是独立服务

当前价格链作为 product-nickname 内的 Module 运行，没有独立 Deployment 或数据库。它的交付不能按“新微服务上线”套模板。

已知操作事实：

- 上传先返回 batch_id，处理是异步；
- 校验、匹配、衍生、输出、去重和批次状态是连续步骤；
- `pricing_final_data` 是结果层，`pricing_final_data_history` 保留写入历史；
- SuperView 直接读取 MySQL 结果；
- 重推通过重新上传，不是原位回滚；
- DDL 必须先于代码部署；
- 网关可能存在新旧后端路由差异；

- 90 天诊断数据清理和审计日志保留策略不同。

因此价格项目验收必须同时检查批次、步骤日志、结果版本、SuperView 查询和生产运行制品，不能只看到 `COMPLETED` 就结束。

6. 产品中心运维的分层检查

运维排查应从服务运行、集成、主数据、投影和消费五层依次取证。服务层查看 Deployment、镜像、profile、健康检查和日志，Running 只证明进程状态。集成层查看 DI 任务、文件、批次和最后更新时间，成功任务行不等于目标业务版本完整。主数据层按业务键检查当前记录、有效期、删除和状态，不能只看总行数。投影层继续检查 DI、Locust、价格结果和查询面的水线、过滤与键覆盖。消费层最终确认规则缓存、门店采集、DTR、订单和报表是否在承诺时点读取目标版本。

层	检查对象	不能由本层单独证明
服务	Deployment、镜像、profile、健康检查、日志	数据已刷新或业务已可用
集成	DI 任务、文件、批次、上游输入、更新时间	目标表已有完整业务版本
主数据	业务键、当前记录、有效期、删除、状态	下游已经读取新版本
投影	DI、Locust、价格结果、水线、过滤、键覆盖	消费结果符合业务目的
消费	规则缓存、门店采集、DTR、订单、报表	历史解释和回滚已经成立

7. 故障分流

现象	先查	再查
新产品页面查不到	上游输入、DI、主数据有效期	查询面过滤与缓存
规则命中错误 SKU	nickname_code、产品范围、缓存版本	规则文件与门店样本
条码扫不到	包装层级、条码投影、设备读取	重复条码和历史版本
订单数量放大	订单 UOM、换算、DTR 映射	库存与报表是否同单位
DTR 无法下单	产品映射、白名单、上下架	价格组、库存、门店范围
新价格 SuperView 不可见	batch 状态、step log、final 表	7 键覆盖、查询视图和网关
历史报表变了	是否重算、读取当前还是快照	旧 UOM/条码/组合是否保留
某服务行为无法从源码解释	镜像与源码分支映射	无源码服务、Active 制品和数据库证据

故障处置必须说明是修源、重建投影、重跑批次、修规则还是回滚制品；不同修复不能互相代替。

8. 已知演进边界

当前知识可以支持设计和排障，但以下问题不能由本手册擅自闭合：

缺口	不能推出的结论
`bz_product*` 与 `t1/t2/t3_*` 双体系边界未最终裁定	新功能可以任选一套写入
产品 SoT 未完全统一	查询最多的一张表就是唯一事实源
`product-poplar`、`product-efs`、`product-wps` 无当前本地源码	可以仅凭运行结果完整重建和回滚
价格链生产合并与上线状态待确认	staging/feature 分支等于生产制品
多 Pod 定时任务互斥未闭环	保留清理和卡死扫描只会执行一次
部分开放查询消费方未定位	下线接口只会影响已发现的调用方

这些边界必须进入项目风险、监控和回滚计划，而不是隐藏在会议结论里。

9. 运营指标

产品运营至少应观察：

面	指标
输入	文件/API 成功、延迟、重复、拒绝原因
主数据	当前/未来/历史行，空键，基数变化
投影	最新水线、覆盖差、失败任务
规则	未匹配 nickname、缓存 trace、目标集合变化
交易	DTR 映射/白名单/价格/库存缺失，订单 WARN
报表	SKU 行、空条码/单位、截P期次和下游延迟
安全	API Key、审计日志、敏感配置暴露

指标只有绑定口径、环境和时点才有意义。当前 staging 数不能直接成为生产告警阈值。

10. 反例

反例一：Pod Running 即上线

运行、数据新鲜、下游可用和业务验收是四种状态。

反例二：批次 FAILED 就重跑

先区分文件/匹配/派生/写入失败。错误输入重跑仍会失败。

反例三：修表替代修链路

直接改 final 表可能让源、历史和下一次批次再次覆盖。

反例四：让最新查询覆盖历史

历史订单和报表必须按当时的产品、单位和价格解释。

11. 本章结论与未知

本章可以支持：

1. 产品项目应按变更类型拆包并沿稳定键交付；
2. 代码、部署、数据、消费和历史证据必须分别验收；
3. 价格链是嵌入式模块，有独立批次、版本与运维顺序；
4. 产品运维应先分层定位，再选择修复方式；
5. 双产品体系、无源码服务和生产状态仍是明确演进边界。

本章仍不回答：

- 产品 SoT 的最终目标结构；
- 所有产品中心服务的源码与运行镜像对应；
- 产品到全部分销、订单和报表消费方的完整映射率；
- 价格链生产发布基线与容量/SLO；
- 任一故障的完整生产事故时间线和恢复演练；
- staging 观察能否外推至生产。

案头入口与收束

项目模板、证据包和发布检查见第 C 卷，事实缺口、维护触发和责任建议见第 D 卷。十二章至此形成从商品语义、稳定身份、单位时间、系统消费，到一致性、变更和项目运维的闭环；以后每次维护都应从来源、时点和未决边界重新验证，而不是把旧结论直接覆盖新事实。

第 A 卷 产品对象与标识

本卷是查询卡，不重新定义业务规则。对象语义以产品中心 Ontology 和概念文档为准，当前运行坐标回 staging 证据。

A.1 对象边界速查

对象	回答什么	典型稳定键	不得替代
产品	哪个 Mars 产品/物料	material_code，必要时 nickname_code	品牌、品类
产品昵称	业务如何引用产品	nickname_code	物料身份
成品/包装	哪个配送或扫描载体	material_code + bar code/UOM	产品全集
S2 组合	哪些单品按什么单位组成商品	product_code	单品 material
UOM	箱、盒、件之间如何换算	product/material + unit_code	订单交易单位
经销商产品	经销商系统中的产品	sold_to + dtr_sku/product mapping	Mars product
报表 SKU	目标报表如何展示/聚合	报表声明的 material/nickname/条码	主数据所有权

A.2 `BzProduct` / `bz_product`

当前实体是 JPA 拉链表，业务理解上的主键是 `zrep_code`；数据库唯一索引是 `material_code + effective_end_timestamp`。两者不能混写。

字段	含义	使用提醒
`material_code`	ERP 物料代码	稳定物料键候选
`zrep_code`	带单位语境的代表码	当前可一对多 material
`zrep_uom`	代表码单位	必须与业务单位语境一起看
`nickname`	业务昵称	不能单独判同一
`nickname_code`	昵称代码	可为空；跨投影不一定携带
`product_category_code/name`	品类	分类不是身份
`alias_brand_code/name`	品牌	分类不是身份
`state`	表状态	业务语义未由本卷裁定
`effective_*`	拉链有效期	当前查询必须带时点
`is_deleted`	逻辑删除	与生命周期不同

A.3 `BzWsFinishedGood` / `bz_ws_finished_good`

成品记录来自 ERP WS 成品清单，适合查询包装与条码，不是“全部产品是否有效”的唯一边界。

字段组	关键字段
身份	`material_code`、`representative_code`、`zrep_code`
单位	`zrep_uom`、`ea_per_tdu`、`mcu_per_tdu`、`rsu_per_tdu`
条码	`tdu_bar_code`、`mcu_bar_code`、`rsu_bar_code`
分类	`product_category`、`brand_flag`、`subsegment`
状态	`material_status`、`effective_*`、`is_deleted`

2026-10-26 staging 当前成品 3,105 行，TDU 空条码 208、RSU 空条码 320。空条码是状态，不是删除指令。

A.4 昵称与条码

对象	当前载体	关键提醒
昵称原事务	`bz_product_nickname`	全量刷新来源，不是对外主查询
昵称字典	`bz_product_nickname_base`	保存唯一昵称词条与代码
名称关系	`bz_product`	当前业务查询表
条码投影	`product_sequoia.di_otr_nickname_barcodes`	当前 1,646 行
成品条码	`bz_ws_finished_good`	按 TDU/MCU/RSU 分开

条码字符串当前存在重复。查询时必须声明：

```
bar code
+ material
+ package unit
+ source table
+ effective time
```

A.5 S2 Header / Line / Unit

层	表	回答
Header	`t2_product_bundle_header_current`	组合产品身份、品牌、品类、生命周期
Line	`t2_product_bundle_line_current`	组合包含哪些单品、数量
Unit	`t2_product_bundle_unit_current`	组合的销售/配送/零售单位

Header 生命周期当前为 Creation、Active、BillAdjmt、Retired。`lifecycle_status`、`is_active` 和拉链有效期是不同字段。

A.6 UOM 卡

单位代码	文档角色	典型业务
CS / TDU	配送箱	物流、仓库、批发
SB / MCU	中间销售箱	零售陈列、促销包装
PCE / RSU	零售件	消费者购买、盘点

当前 S2 校验要求：

- 必须包含 TDU 与 RSU；
- TDU 换算系数为 1；
- MCU 可选；
- RSU 系数不得小于 MCU；
- 有 MCU 时，RSU 系数必须是其整数倍；
- TDU 长宽高必填，条码不超过 100 位。

这些是 S2 写入校验，不代表所有订单、库存和报表已经自动使用同一单位。

A.7 当前标识基数

2026-10-26 staging：

载体	精确基数
当前 `bz_product`	3,115
distinct material	3,115
distinct zrep	1,355
distinct nickname_code	1,247
空 nickname_code	320
当前成品 material	3,105
S2 Header / Line / Unit	24 / 59 / 51

2026-10-27 按 material 对账：3,115 个产品中 10 个无当前成品匹配；3,105 个当前成品全部匹配产品。

A.8 使用规则

1. 先声明对象和粒度，再选键。
2. 名称用于搜索，不用于最终同一性。
3. 条码前必须判断包装层级。
4. UOM 必须进入订单、库存和报表连接条件。
5. 历史解释必须固定业务时点。
6. 空值与不适用不得默认为 0 或 false。

A.9 未决

- `bz_product*` 与 `t1/t2/t3_*` 的最终 SoT 边界。
- `bz_product.state` 的完整业务字典。
- 条码重复的合法范围与拆分规则。
- UOM 在不同下游的最终权威单位。
- 10 个产品→成品单向缺失的业务解释。

第 B 卷 产品系统与数据链

B.1 运行面

2026-10-26 staging 的六个产品中心 Deployment 均为 1/1：

运行面	主要职责
product-nickname	产品、昵称、分类、属性、S2、价格模块
product-sequoia	DI、DAQ/DTR、产品投影与数据分片
product-locust	实体定义、开放查询、动态 API
product-poplar	实体引擎运行面
product-efs	文件与产品单位能力
product-wps	产品前端运行面

Deployment Running 只证明进程就绪。当前未有 `product-poplar`、`product-efs`、`product-wps` 的当前本地源码，因此不能证明可从源码完整重建。

B.2 产品主数据链

```
ERP / MDP
  → product-sequoia DI
  → product_nickname 处理与投影
  → bz_product / bz_ws_finished_good / t1-t3
  → product-locust / DI / Feign
  → SDRS / DTR / 门店 / 报表 / SuperView
```

两条已登记主线：

1. ERP 摄入：MDP → product-sequoia → `bz_ws_finished_good` → `t1_product_individual_current` → `bz_product` → `bz_product_attr`。
2. 昵称维护：用户文件 → product-sequoia → Feign 回调 → `bz_product_nickname` → `bz_product` → `bz_product_attr`。

链路顺序来自产品中心 README；当前仍缺每一步的逐任务新鲜度与失败率。

B.3 核心 API

产品昵称

能力	入口
产品昵称查询/维护	`ProductNikeNameController`
DI 回调/同步	product-sequoia + product-nickname 集成
下游批量取昵称	`/product/nickname/all`

S2

动作	API
创建	`POST /product/s2`
更新	`PUT /product/s2`
详情	`GET /product/s2/get/detail`
分页	`GET /product/s2/pageable`
取编码	`GET /product/s2/get/code`
子产品	`GET /product/s2/children/list`

S2 API 请求体包含 Header、children 与 units。项目必须按三层保存和验证，不能只传一个产品编码。

开放查询

product-locust 的统一查询入口为 `/query/entity/data`，按 `api_definition` / `api_configuration` 动态路由到 SQL 或 MongoDB。当前 `api_definition` 有 5 条。

查询前必须确认：

1. 使用哪个 API 配置；
2. D1 还是 DyMongo；
3. 字段定义版本；
4. 过滤和时点；
5. 调用应用的授权。

B.4 价格链

价格链是 product-nickname 内的 Module，不是独立服务或 Deployment。

```
Horde Excel
  → 上传/校验
  → pricing_staging_data
  → 产品匹配与 13 维属性
  → 衍生公式
  → 46 字段组装
  → 7 键去重
  → pricing_final_data
  → pricing_final_data_history
  → SuperView
```

批次追踪键是 `batch_id`。失败时先看 `pricing_batch_status`，再看 `pricing_pipeline_step_log`。

B.5 下游消费

消费方	读取方式	关键风险
SDRS	调产品昵称 API 补产品属性	名称/编码粒度
fdw-fns	建规则缓存时回查产品	DI 与主表范围差
门店报表	直读 product_sequoia 成品投影	material/条码/单位
完美门店	产品 API 或报表投影	nickname/material 口径
DTR	Mars product out 与经销商映射	映射、上下架、价格、库存
订单/库存	产品键 + UOM/经销商产品	交易单位与库存结批
SuperView	直读价格结果表	batch、7 键和历史版本

B.6 证据采集模板

```
环境:
观察时间:
服务 / Deployment:
镜像 / profile:
代码仓 / 分支 / 提交:
入口 API / 任务 / 场景:
输入键:
输出表 / API:
最后更新时间:
未核验项:
```

B.7 只读检查顺序

1. Deployment 就绪、镜像与 profile。
2. 上游 DI 任务、批次和最后成功时间。
3. 当前主数据记录、状态和有效期。
4. DI/开放查询/价格结果的水线。
5. 下游读取的是 API、投影还是直读表。
6. 目标业务样本是否命中。

B.8 运行缺口

- product-sequoia 调度触发源与失败重试未逐项闭环。
- product-locust 查询消费方和 SQL/Mongo 权限边界未取全。

- 价格链生产合并与上线状态待确认。
- 多 Pod 定时任务互斥性未闭环。
- 部分服务 SLO、慢查询、备份恢复和告警阈值未登记。

第 C 卷 产品项目工作台

C.1 通用项目卡

项	必填
变更类型	新品、昵称、条码、包装、UOM、S2、价格、退市、SoT、修数
业务键	material / nickname_code / zrep / barcode / S2 code / batch_id
旧新值	源值、目标值和生效时间
范围	应用、渠道、客户、门店、报表
Owner	业务、数据、应用、测试、运维
证据	代码、部署、配置、数据、测试、下游
回退	配置/代码/数据/批次恢复顺序

C.2 新品引入

检查顺序：

```
上游 material
  → product DI
  → bz_product/成品/属性
  → 开放查询
  → 分销规则产品范围
  → DTR 映射/可售/价格/库存
  → 门店采集
  → 订单/库存/报表
```

通过标准：

- 同一 material 在目标表面均可查；
- 条码与包装单位正确；
- 规则缓存出现目标；
- 经销商映射和可售按目标范围成立；
- 门店实际采集可识别；
- 报表目标行与单位可复算。

C.3 昵称/条码修正

检查	证据
旧新 nickname_code	`bz_product*`
名称在 DI/规则中的读法	DI 投影、规则样例
条码所属包装层	成品/单位/API
扫描与识别	目标设备/IR/报表
历史查询	旧名称/旧条码是否保留

不得做全库名称替换。

C.4 包装与 UOM

项目必须提交一条完整换算样例：

```
1 TDU = N MCU = M RSU
```

并选择：

- 一条新订单；
- 一条历史订单；
- 一条库存；
- 一条报表聚合；
- 一条 DTR 经销商产品映射。

分别复算数量、金额和单位。

C.5 S2 组合

最低用例：

用例	断言
三层创建	Header/Line/Unit 完整
生命周期	Creation/Active/BillAdjmt/Retired 合法
单位	TDU/RSU 必然，MCU 可选且换算合法
组合拆单	子产品、qty、unit 正确
查询	组合详情和子产品可追
历史	更新后旧版本可按时点取回

C.6 退市/停用

必须区分：

- 生命周期退市；
- `is_active`；
- 记录有效期；
- 逻辑删除；
- 经销商上下架；
- 价格未来版本。

检查新使用是否阻断，同时证明历史订单、库存和报表仍可解释。

C.7 价格方案

UAT 场景组：

组	覆盖
上传校验	UAT-PRICE-UPLOAD-01~05
产品匹配/衍生	UAT-PRICE-DERIVE-01~06
输出/版本	UAT-PRICE-OUT-01~05
批次/血缘	UAT-PRICE-BATCH-01~05
SuperView	UAT-PRICE-SV-01~05
认证/审计	UAT-PRICE-AUTH-01~04

每个执行记录至少保存 `batch_id`、SQL/API、结果、缺陷、执行人和日期。

C.8 数据修复

修复前：

1. 证明差异不是单位、时点或投影；
2. 找根因和影响键；
3. 定义只修目标批次还是重算投影；
4. 做幂等与前后快照；
5. 评估历史订单/报表；
6. 指定回退和复核人。

直接更新目标表只在被批准且可追溯时使用，不能替代上游修复。

C.9 证据包

```
需求与范围
影响图
代码/分支/提交
Deployment/镜像/profile
DI/场景/制品
DDL 与配置
修改前后数据
batch_id / request_id
正反例、历史、重跑
下游回读
回滚演练
残留 OQ
```

C.10 发布前检查

- 业务键无歧义；
- 名称没有被当唯一键；
- 包装单位已确认；
- DDL 已先行；
- 价格批次可追；
- 下游读取的投影已确认；
- 历史样本可解释；
- 回滚和值守责任人已明确。

第 D 卷 产品事实缺口、术语与维护

D.1 当前高优先级缺口

编号	缺口	影响
P-OQ-01	`bz_product*` 与 `t1/t2/t3_*` 最终 SoT	新功能可能写错体系
P-OQ-02	UOM 最终权威层	订单、库存、报表使用不同单位
P-OQ-03	价格链生产状态	staging/feature 分支不能写为已上线
P-OQ-04	无当前源码的运行服务	无法完整重建、审计和回滚
P-OQ-05	10 个产品→成品 material 单向缺失	原因和责任未定
P-OQ-06	条码重复和空值	扫描与识别解释不完整
P-OQ-07	开放查询消费方未全量定位	接口下线影响不确定
P-OQ-08	DI 调度、重试和新鲜度承诺	数据时延无法承诺

D.2 术语与禁止混用

规范概念	不是
Product / 产品	品牌、品类、昵称
material_code	nickname、barcode
zrep_code + UOM	不带单位的物料代表码
nickname_code	自然名称
barcode + package unit	产品主键
S2 Header/Line/Unit	单个 material
经销商产品	Mars 产品
产品存在	可售、有价格、有库存、可下单
当前记录	历史版本或未来版本

D.3 事实分级

等级	能证明
文档	业务/系统设计声明
代码	当前源码声明或实现，需确认分支
Deployment	某环境运行镜像和就绪
Active 制品	某计算/规则制品在当前平台启用
staging DB	指定时点的数据观察
下游回读	指定消费链在该时点给出结果
业务验收	被授权人确认目标业务结果

等级之间不可跳跃。

D.4 当前数值失效触发

以下观察默认在数据变化、制品变化或过观察日后失效：

- `bz_product` 3,115、distinct zrep 1,355、nickname 1,247；

- 当前成品 3,105；
- TDU/RSU 空条码 208/320；
- S2 24/59/51；
- DI 条码投影 1,646；
- product 与成品双向 10/0 缺失；
- 规则 217 个 nickname 的名称覆盖 143/217；
- 六个产品中心 Deployment 就绪。

D.5 维护触发

发生以下事件时必须复核本手册：

1. 产品 SoT 或 S2 模型变更；
2. 新增/删除产品中心服务；
3. 价格链发布或运维方式变化；
4. DI 投影、调度和表结构变化；
5. 规则产品范围或产品 API 变化；
6. DTR 产品映射、订单单位或库存模型变化；
7. 关键 OQ 关闭；
8. staging/生产发布基线变化。

D.6 查询前检查

- 决定问的是物料、包装、昵称、组合、经销商产品还是报表 SKU；
- 选择稳定键；
- 声明删除、状态、有效期；
- 对齐源和目标的观察时间；
- 检查单位；
- 把名称仅作为辅助；
- 记录环境和查询口径。

D.7 维护责任建议

范围	责任
产品身份与单位	产品数据 Owner
服务/API/DI/部署	产品应用 Owner
价格链	价格业务 + 应用/运维 Owner
DTR/订单/库存	对应系统 Owner
报表	报表数据 Owner
手册结构	产品域 Ontology/编辑责任

责任角色在本卷是分工建议；具名 Owner 仍以项目/运行台账为准。

D.8 事实声明纪律

正文中的“已实现”“已部署”“已生效”“已验收”必须分别落到代码、Deployment、业务生效和验收证据。没有证据时写：

```
未取得
未核验
仅 staging 观察
待业务裁定
```

不得用推测填补缺口。

生成边界

本文件是装配成品，不新增业务事实。修改章节、案头卷或 manifest 后必须重新运行生成器；事实冲突回 source_docs 和 claim ledger。